HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

End-to-End Ensemble Learning Architecture for Intrusion Attack Classification

TRIEU KINH QUOC
quoc.tk225524@sis.hust.edu.vn

Program: Data Science and Artificial Intelligence

Supervisor: Associate Professor Pham Van Hai

Department: Computer Science

School: School of Information and Communications Technology

HANOI, 06/2026

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

End-to-End Ensemble Learning Architecture for Intrusion Attack Classification

TRIEU KINH QUOC
quoc.tk225524@sis.hust.edu.vn

Program: Data Science and Artificial Intelligence

Supervisor: Associate Professor Pham Van Hai _____

Signature

Department: Computer Science

School: School of Information and Communications Technology

HANOI, 06/2026

ACKNOWLEDGMENT

My four years at Hanoi University of Science and Technology (HUST) have been a journey with precious memories and experiences. From a freshman, I have now accumulated the foundational knowledge and skills to confidently step into the world, ready to contribute to society. This graduation thesis is not merely the conclusion of a course, but a milestone marking my personal growth.

To complete this thesis, I would like to express my deepest gratitude to my instructor, Assoc. Prof. Pham Van Hai. His dedicated guidance, insightful feedback, and rigorous standards have helped me continuously improve my thesis and maintain a serious work ethic. I sincerely wish him abundant health and further success in his noble teaching career.

Furthermore, I would like to express my sincere appreciation to M.Sc. Nguyen Dinh Nga (University of Transport and Communications) and Mr. Duc (IBM Corporation) for their enthusiastic support regarding specialized knowledge as well as technical infrastructure deployment. Their companionship has been a tremendous motivation, helping me complete this project to the fullest.

Moreover, I would like to send my profound gratitude to my family who gave me the opportunity to study in this wonderful environment. I also want to thank my companions over the past four years, who have always listened to and shared my joys in both academic and everyday life.

My journey at HUST has not only equipped me with a wealth of knowledge but also given me the opportunity to immerse myself in the GDG-HUST community. This environment has helped me hone essential soft skills–which will undoubtedly be a great asset to my future career path. As I bring this brilliant chapter to a close, I will always carry my pride and engrave in my heart the saying:

"Once a HUSTer, always a HUSTer."

ABSTRACT

Network-based Intrusion Detection Systems (NIDSs) need to accurately identify increasingly sophisticated attacks from large-scale, imbalanced, and temporally evolving network flows. The thesis proposes an end-to-end ensemble learning architecture for multi-class intrusion attack classification in IoT and IIoT environments. The proposed framework combines two complementary base learners. The first branch constructs a flow-based graph and applies Graph Attention Network embeddings [1] with XGBoost [2] to capture spatial relationships among network flows. The second branch transforms chronological traffic into sliding windows and uses residual convolutional blocks, Squeeze-and-Excitation recalibration, BiLSTM layers, and attention to model temporal attack behavior. A meta-learner then fuses the predictive distributions of both branches to obtain the final classification output. Experiments are conducted on three use cases from IoT-IDS-Zwave-2025 [3] and CIC IIoT 2025 [4], covering HTTP-based DDoS attacks, application-based DDoS attacks, and industrial IoT attack traffic. The results show that the proposed ensemble is competitive with traditional machine-learning, graph-based, and recent ensemble IDS baselines, achieving strong macro-F1 performance and improved robustness in challenging network traffic scenarios. In particular, the meta-learner benefits from the complementarity between spatial graph representations and temporal sequence representations, making the architecture suitable for heterogeneous intrusion detection settings.

Student

(Signature and full name)

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION 1

1.1 Problem Statement 1

1.2 Background and Problems of Research 2

1.3 Research Objectives and Conceptual Framework 4

1.4 Contributions 5

1.5 Organization of Thesis 5

CHAPTER 2. THEORETICAL BACKGROUND 8

2.1 Scope of Research 8

2.2 Related Work 9

2.3 Convolutional Neural Networks 10

2.4 Squeeze-and-Excitation Networks 11

2.5 Bidirectional Long Short-Term Memory 11

2.6 Graph Attention Network 12

2.7 XGBoost (eXtreme Gradient Boosting) 13

2.8 Ensemble Learning 13

CHAPTER 3. METHODOLOGY 15

3.1 Problem Formulation and Notation 15

3.2 Overall Architecture 15

3.3 Flow-Based Graph Construction 16

3.4 Base Learner 1: GAT Embedding - XGBoost 17

3.5 Base Learner 2: Residual CNN–BiLSTM 19

3.6 Meta-Learner 21

CHAPTER 4. EVALUATION METHODS 23
4.1 Evaluation Dataset 23
4.1.1 BCCC-IoT-IDS-Zwave-2025 Dataset 23
4.1.2 CIC IIoT 2025 Dataset 25
4.2 Data Preprocessing 26
4.3 Evaluation Metrics 27
4.4 Experimental Environment and Setup 28
CHAPTER 5. ABLATION STUDY 30
5.1 Ablation Study for the Residual CNN-BiLSTM Base Learner 31
5.2 Ablation Study for the GAT Embedding + XGBoost Base Learner 32
5.3 Ablation Study for the Meta-Learner Architecture 33
CHAPTER 6. EXPERIMENTAL RESULTS 35
6.1 Base Learners 35
6.1.1 Simulation Method 35
6.1.2 Use Case 1: HTTP-based Attack - IoT-IDS-Zwave-2025 36
6.1.3 Use Case 2: Application-based Attack - IoT-IDS-Zwave-2025 37
6.1.4 Use Case 3: CIC IIoT 2025 38
6.2 Meta Learner 38
6.2.1 Simulation Method 38
6.2.2 Results across Use Cases 39
CHAPTER 7. Case Study: Simulated Real-Time Inference Deployment ... 42
7.1 Overall 42
7.2 Deployment Diagram 42
7.2.1 Data Generator Simulation 43
7.2.2 Kafka 44
7.2.3 Spark Streaming 45

7.2.4 Ensemble Inference 46
7.2.5 InfluxDB 47
7.2.6 Grafana 48
7.3 Dashboard Visualization 49
7.3.1 Chart: Total Inferred Flows 50
7.3.2 Chart: Top Threat Clusters 50
7.3.3 Chart: Benign vs. Malicious 51
7.3.4 Chart: Malicious Flow Distribution 52
7.3.5 Chart: Daily Total Network Threats 52
7.3.6 Chart: Daily Threat Distribution by Type 53
7.4 Deployment Conclusion 53
CHAPTER 8. CONCLUSIONS 55
8.1 Limitations and Future Work 55
8.2 Conclusion 55
REFERENCE 60

LIST OF FIGURES

Figure 1.1 Most common types of cyberattacks. 2
Figure 1.2 Trend of deep learning in IDS research. 3

Figure 2.1 Common workflow of DL-IDS. 9
Figure 2.2 Convolutional Neural Networks. 10
Figure 2.3 Squeeze-and-Excitation block. 11
Figure 2.4 BiLSTM. 12
Figure 2.5 GAT mechanism. 13

Figure 3.1 Overall architecture of the proposed ensemble intrusion detection framework. 16
Figure 3.2 Spatial base learner based on GAT embeddings and XGBoost classification. 17
Figure 3.3 Temporal base learner based on residual CNN blocks and BiLSTM attention. 19
Figure 3.4 Residual block used in the temporal base learner. 19

Figure 4.1 HTTP-Based Attacks - IoT-IDS-Zwave-2025: Class Distribution. 24
Figure 4.2 Application-Based Attacks - IoT-IDS-Zwave-2025: Class Distribution. 25
Figure 4.3 CIC IIoT 2025: Class Distribution. 26

Figure 7.1 Deployment diagram of the proposed ensemble architecture. 43
Figure 7.2 Dashboard Visualization by Grafana 49
Figure 7.3 Total inferred flows displayed in the Grafana dashboard. . . . 50
Figure 7.4 Top threat clusters detected by the Grafana dashboard. 50
Figure 7.5 Benign and malicious flow proportions in the Grafana dashboard. 51
Figure 7.6 Distribution of malicious flows by attack category in the Grafana dashboard. 52
Figure 7.7 Daily total network threats visualized in the Grafana dashboard. 52
Figure 7.8 Daily threat distribution by attack type in the Grafana dashboard. 53

LIST OF TABLES

Table 3.1 Configuration of residual blocks in the temporal base learner. 20
Table 3.2 Configuration of the BiLSTM-attention module. 21

Table 5.1 Ablation Study Results for Residual CNN-BiLSTM Variations across Three Use Cases 31
Table 5.2 Ablation Study Results for GAT and XGBoost Architectures across Three Use Cases 32
Table 5.3 Ablation Study Results for Meta-Learner Architectures across Two Use Cases 33

Table 6.1 Performance Evaluation of Intrusion Detection Models - Use Case 1 36
Table 6.2 Performance Evaluation of Intrusion Detection Models - Use Case 2 37
Table 6.3 Performance Evaluation of Intrusion Detection Models - Use Case 3 38
Table 6.4 Performance comparison of Ensemble Learning models - Use Case 1 39
Table 6.5 Performance comparison of Ensemble Learning models - Use Case 2 39
Table 6.6 Performance comparison of Ensemble Learning models - Use Case 3 40

LIST OF ALGORITHMS

Algorithm 3.1 Pseudo-code for flow-based graph construction. 17

Algorithm 7.1 Real-time Traffic Simulation with Configurable Speed Factor 44

LIST OF ABBREVIATIONS

BiLSTM Bidirectional Long Short-Term Memory

CNN Convolutional Neural Network

GAT Graph Attention Network

IDS Intrusion Detection System

SE Block Squeeze-and-Excitation Block

CHAPTER 1. INTRODUCTION

1.1 Problem Statement

The development of effective Intrusion Detection Systems (IDS) remains a critical challenge amidst the rapid expansion of information technology infrastructure [5], [6]. According to the National Cybersecurity Association, information systems in Vietnam encountered approximately 552,000 cyberattacks in 2025. Despite a 19.38% reduction in the total volume of attacks compared to 2024, the resultant financial and operational damages have escalated significantly. Furthermore, the proportion of enterprises exposed to cyber threats surged to 52.30%, up from 46.15% in the preceding year. This paradigm shift indicates that adversaries are increasingly prioritizing the sophistication. As attack strategies become more complex and dynamic, there is a need to engineer robust, adaptive IDS capable of learning from the most current threat trends [7].

To mitigate these risks, 51.56% of enterprises in Vietnam have conducted cybersecurity trainings for personnel, while 76.35% have implemented data backup protocols to counteract potential post-attack consequences. However, reports also indicate that a concerning 51.87% of organizations have yet to deploy standard anti-malware platforms. This vulnerability highlights an urgent necessity for the development of comprehensive, end-to-end security systems tailored for real-world enterprise applications. Consequently, designing an advanced IDS capable of accurately recognizing and classifying network traffic logs into specific cyber-attack sub-categories or benign is of importance [7].

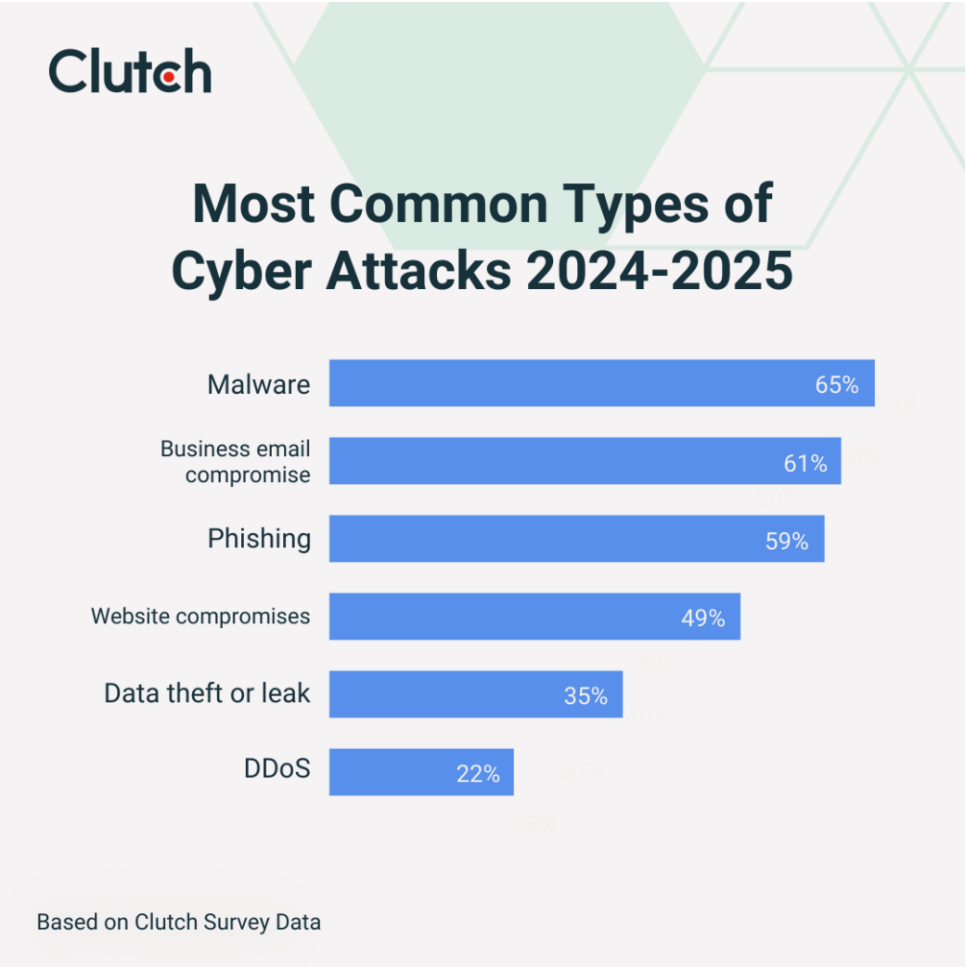

Figure 1.1: Most common types of cyberattacks [8].

Given the destructive nature of modern cyberattacks, contemporary systems demand high detection accuracy and the ability to precisely identify attack variants. This exactness is crucial for enabling system administrators to formulate appropriate and timely defensive measures. Addressing this requirement, this thesis proposes a novel framework utilizing two high-performance deep learning models designed to analyze both the spatial and temporal dimensions of network traffic. By ensembling these models, the proposed system can effectively classify node-to-node network communications into distinct attack categories. Furthermore, to ensure scalability and real-time processing capabilities in enterprise environments, the entire architecture is containerized using Docker and integrates a robust large-scale streaming data pipeline facilitated by Apache Kafka and Apache Spark.

1.2 Background and Problems of Research

Intrusion Detection Systems (IDS) are generally categorized into two primary paradigms: **Host-based IDS (HIDS)** and **Network-based IDS (NIDS)**. While HIDS focuses on analyzing internal log files within a single host machine, NIDS is tasked with inspecting and classifying network traffic logs exchanged between interacting

hosts. **This research specifically concentrates on the implementation of NIDS.**

In terms of the IDS domain, Deep Learning (DL) has emerged as a novel and highly promising frontier. According to a recent 2025 survey on DL applications in IDS, there is a consistently upward trend in its publication. This growth is primarily driven by DL's substantial performance improvements over traditional machine learning methodologies, albeit at the cost of increased computational complexity [9].

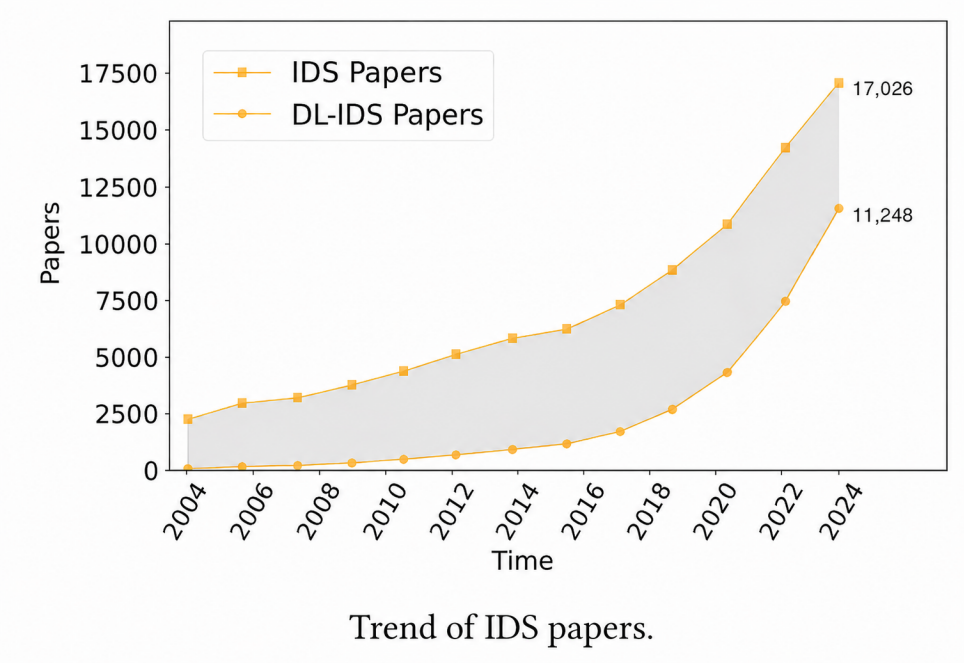

Figure 1.2: Trend of deep learning in IDS research [9].

Deep Learning has achieved significant milestones in terms of network-based traffic logs (NIDS). In 2022, Zhao et al. introduced MT-FlowFormer, a semi-supervised deep learning framework predicated on the Transformer architecture, designed to classify encrypted network traffic [10]. Similarly, Amna Naeem et al. proposed a hybrid CNN–BiLSTM architecture that achieved high accuracy in botnet attack detection [11]. Recent IoT and IIoT studies further extend deep NIDS research through deep transfer learning, multitask multiview learning, and DDoS-specific deep models [12], [13], [14]. More recently, the focus of DL-IDS literature has shifted toward Graph Neural Networks (GNNs) due to their robust capability to capture spatial topologies—a core characteristic inherent to network intrusion detection. A notable contribution in this area is E-GraphSAGE, which enhances the standard GraphSAGE architecture by incorporating edge features [15]. Building upon this, NetVigil is a classification system combining E-GraphSAGE with contrastive learning paradigms to augment detection efficacy [16]. Furthermore, in

a recent study, Villegas-Ch et al. proposed the integration of GNNs with dynamic graph modeling to detect and classify attacks within IoT network ecosystems [17]. Centrality-aware graph deep-learning frameworks have also been proposed for IoT intrusion detection, further reinforcing the usefulness of graph-structured traffic representations [18].

Alongside standalone deep learning models, Ensemble Learning techniques have also demonstrated substantial efficacy in NIDS by ensembling the strengths of diverse base classifiers. For instance, xIDS introduced an ensemble approach that integrates tree-based algorithms (LightGBM, GBM, Bagging, XGBoost, CatBoost) with deep learning models (LSTM, GRU) through a meta-learner to yield highly reliable and accurate predictions [19]. Additionally, Nugroho proposed a two-layer Stacking Ensemble Learning framework—utilizing XGBoost, LightGBM, CatBoost, and AdaBoost as base learners with a Random Forest meta-learner—to effectively balance prediction bias and variance [20]. Recent studies also report ensemble IDS designs and IoT-edge ensemble stacking, supporting the continued relevance of ensemble fusion in practical intrusion detection systems [21], [22].

Despite these advancements, existing architectural models are predominantly evaluated on outdated datasets. Furthermore, these deep learning models typically operate in isolation and struggle to mitigate the phenomenon of concept drift—the continuous evolution of attack vectors and behavioral patterns over time. Identifying this critical research gap, this study proposes a novel framework comprising two distinct deep learning models dedicated to analyzing both the temporal and spatial dimensions of network traffic logs. To address dynamic threat landscapes, both models are engineered with robust anti-concept drift mechanisms and are strategically ensembled via a meta-learner to maximize overall detection performance and systemic resilience.

1.3 Research Objectives and Conceptual Framework

The objective of this thesis is to engineer and optimize a high-quality classification Ensemble Learning system capable of classifying network traffic flows between interacting Internet of Things (IoT) devices, thereby accurately categorizing corresponding attack vectors. To overcome the limitations of existing works discussed in Section 1.2, this research proposes the following methodological advancements:

- **Addressing the Limitation of Outdated Datasets:** The proposed models will be rigorously evaluated using the recently published datasets.
- **Spatio-Temporal Traffic Analysis via Hybrid Architecture:** This thesis introduces an advanced hybrid framework integrating a Residual CNN–BiLSTM

architecture, Graph Attention Network (GAT) embeddings, and an XGBoost classifier. This structural design facilitates a comprehensive extraction and analysis of network flows across both spatial and temporal dimensions. The predictive capabilities of these base models are fused through an overarching Meta-Learner, thereby significantly enhancing the overall detection efficacy and systemic accuracy.

- **Mitigating Concept Drift in Attack Signatures:** To deal with the challenge of concept drift—where the statistical properties and behavioral patterns of attack traffic logs continuously evolve within an attack—the foundational CNN–BiLSTM base model [11] is strategically augmented with Residual connections and an Attention mechanism. These structural enhancements ensure the model maintains robust feature extraction capabilities and adapts dynamically to temporal shifts spanning the duration of an attack.

1.4 Contributions

The main contributions of this thesis are summarized as follows:

- **Design a flow-as-node graph construction strategy and a GAT–XGBoost spatial learner** that captures relationships among temporally related flows while avoiding temporal leakage. The graph-enhanced learner achieves the strongest Macro F1 and AUC-ROC in the HTTP-based DDoS attack classification.
- **Develop a Residual CNN–BiLSTM temporal learner** with sliding-window sequence modeling, residual feature extraction, and attention-based aggregation. This learner consistently outperforms its ablated variants and achieves the best Macro F1 and AUC-ROC in the more challenging CIC IIoT 2025 dataset.
- **Propose a stacked ensemble framework** that fuses the predictive distributions of the spatial and temporal learners through logistic-regression and XGBoost meta-learners. The ensemble improves robustness by exploiting complementary spatial and temporal evidence, especially when attack classes are imbalanced or harder to separate.

1.5 Organization of Thesis

To present the research systematically and logically, this thesis is structured into eight distinct chapters, outlined as follows:

- **Chapter 1: Introduction.** This chapter provides a comprehensive overview of the problem's urgency, reviews the existing literature, and identifies the current methodological limitations within the field. The core objectives and primary

contributions of this research are also stated.

- **Chapter 2: Theoretical Background.** This chapter states the research scope and establishes the foundational theories that construct the proposed architectural models. It presents an in-depth analysis of the Graph Attention Network (GAT) and XGBoost, alongside the fundamental mechanisms of the components utilized within the Residual CNN–BiLSTM model, including the Residual Block, Squeeze-and-Excitation (SE) Network, Convolutional Neural Networks (CNN), Bidirectional LSTM (BiLSTM), and Ensemble Learning paradigms.
- **Chapter 3: Methodology.** This chapter elaborates on the proposed architectural framework, partitioned into three main parts: GAT Embedding - XGBoost, Residual CNN–BiLSTM, and Meta-Learner Model. Each part specifically details the architectural design process, hyperparameter tuning strategies, and training methodologies employed to achieve optimal predictive performance.
- **Chapter 4: Evaluation Methods.** This chapter presents a detailed overview of the evaluation datasets, which serve as the primary benchmarks for model evaluation. It details the datasets' statistical characteristics and the data splitting, preprocessing strategies for experimental assessment in Chapter 5 and Chapter 6. It also states the evaluation metrics and environmental setup for the evaluation.
- **Chapter 5: Ablation Study.** This chapter focuses on conducting rigorous ablation studies for both the base learners and the meta-learner. Based on these investigations, critical conclusions are drawn regarding the efficacy, functional contribution, and necessity of each distinct component within the proposed system architecture.
- **Chapter 6: Experimental Results.** The outcomes and comparative analysis against established baselines and current adapted State-of-the-Art (SOTA) architectures are presented in this chapter. It outlines the reasons for selecting comparative baselines, details the experimental configurations, and provides comprehensive results of the comparative performance metrics.
- **Chapter 7: Case Study: Simulated Real-Time Inference Deployment.** This chapter describes the practical deployment phase, detailing the containerization of the proposed architecture using Docker and the implementation of a large-scale streaming data pipeline facilitated by Apache Kafka and Apache Spark. Furthermore, it provides visual demonstrations of the operational system, verifying its readiness for end-to-end production environments.

- **Chapter 8: Conclusions.** The final chapter summarizes the primary theoretical and practical contributions of the thesis. Additionally, it objectively acknowledges the inherent limitations of the current system and proposes potential future research directions and architectural expansions for future development.

CHAPTER 2. THEORETICAL BACKGROUND

2.1 Scope of Research

According to the comprehensive survey [9], an Intrusion Detection System (IDS) is formally defined as *"a software or hardware system to automate the process of intrusion detection"*. Based on the fundamental nature of the underlying data sources and specific analytical objectives, IDS architectures are broadly categorized into three primary paradigms:

- **Network-based IDS (NIDS):** An IDS paradigm where the primary data sources consist of network traffic exchanged between hosts. Strategically deployed at the edge or key routing nodes of a network, NIDS provides holistic security oversight across the entire computing infrastructure utilizing aggregated network data. However, its efficacy diminishes when addressing intra-host anomalies, and it faces significant analytical limitations when processing encrypted network payloads.
- **Host-based IDS (HIDS):** An IDS configured to monitor system-level events and internal activities strictly within individual hosts. HIDS facilitates highly granular, comprehensive anomaly detection on the target machine and remains entirely unimpeded by network-level encryption, as data decryption inherently occurs locally at the host level.
- **Provenance-based IDS (PIDS):** A specialized evolution of HIDS that leverages data provenance as its core analytical source. By analyzing intact, chronological event trails and executing rigorous causality analysis, PIDS has proven highly effective in mitigating advanced, multi-stage cyberattacks while substantially reducing false positive alarm rates.

Within the established boundaries of this study, the primary focus is **strictly restricted to the NIDS paradigm—specifically, the modeling and classification of network traffic flows.** Alternative detection architecture, notably HIDS and PIDS, fall explicitly outside the defined scope of this thesis.

Furthermore, also stated in [9], the standard operational workflow of a holistic IDS encompasses seven sequential stages: Raw Data, Collection, Storage, Parsing, Summarization, Detection, and Investigation. This complete, end-to-end processing pipeline is visually illustrated in the following figure:

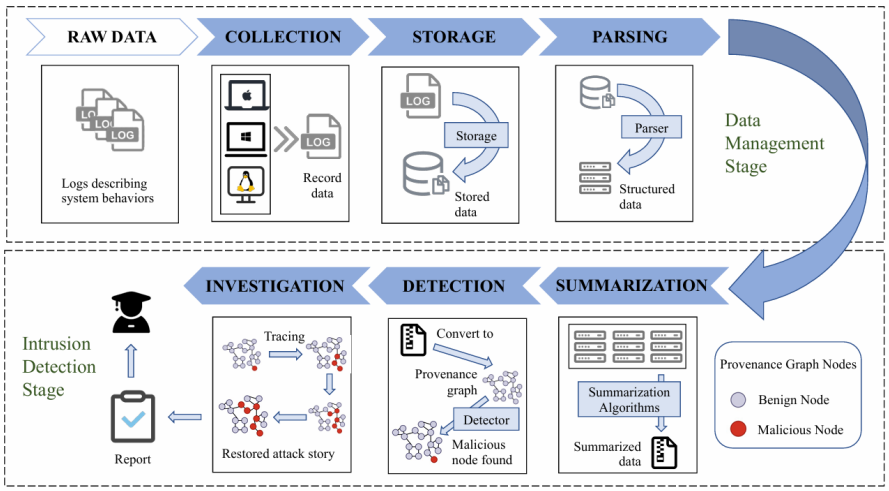

Figure 2.1: Common workflow of DL-IDS [9].

In the context of this study, because the experimental data has been pre-collected and standardized into structurally uniform CSV files, the primary focus is **narrowed exclusively to the fifth stage: Detection.** Consequently, data aggregation and the final stage, Investigation, fall outside the scope of this research.

2.2 Related Work

Prior NIDS research has evolved from conventional machine-learning pipelines on benchmark datasets toward deep, graph-based, and ensemble architectures. Early and widely used datasets such as UNSW-NB15, CICIDS2017, Bot-IoT, and TON-IoT established realistic traffic corpora for evaluating intrusion classifiers under heterogeneous attack scenarios [23], [24], [25], [26]. On the modeling side, recurrent and convolutional deep networks have been adopted to learn nonlinear temporal behavior from packet or flow sequences [11], [27], [28]. Recent IoT and IIoT intrusion-detection studies further extend this direction through deep transfer learning, multitask multi-view learning, and DDoS-oriented deep models [12], [13], [14]. Graph neural networks have also been introduced to represent network communications as structured relational data and to capture spatial dependencies between entities or flows [1], [15], [17], [29], [30], [31], while centrality-aware graph deep-learning models provide additional evidence that graph-structured traffic representations are useful for IoT intrusion detection [18]. In parallel, boosted-tree and stacking ensembles remain competitive because they combine complementary learners and reduce variance in imbalanced traffic settings [2], [19], [20], [32], [33]. Recent ensemble-oriented IDS studies, including subspace-clustering ensembles and IoT-edge ensemble stacking, further support the relevance of model fusion in practical deployment settings [21], [22]. These studies collectively motivate a compact spatial–temporal ensemble design that preserves graph-level relationships,

sequential flow dynamics, and robust meta-level decision fusion for recent IoT and IIoT intrusion datasets [3], [4]. The cited recent works are used to contextualize related modeling trends rather than as direct experimental baselines, since they rely on different datasets, traffic representations, and evaluation protocols.

2.3 Convolutional Neural Networks

Convolutional Neural Networks (CNN) represent a specialized class of deep learning models explicitly engineered to process data possessing a grid-like topology, most notably image matrices or time-series sequences. The core computational efficacy of the CNN architecture relies upon two fundamental operations: convolution and pooling. Convolutional layers utilize learnable filter matrices (kernels) that systematically slide across the input data to autonomously extract local features, thereby enabling the model to recognize complex underlying patterns. Subsequently, the pooling layer—typically implementing a Max Pooling operation—serves to downsample the spatial dimensionality of these representations. This process effectively filters extraneous noise, enhances translation invariance, and significantly mitigates the risk of model overfitting.

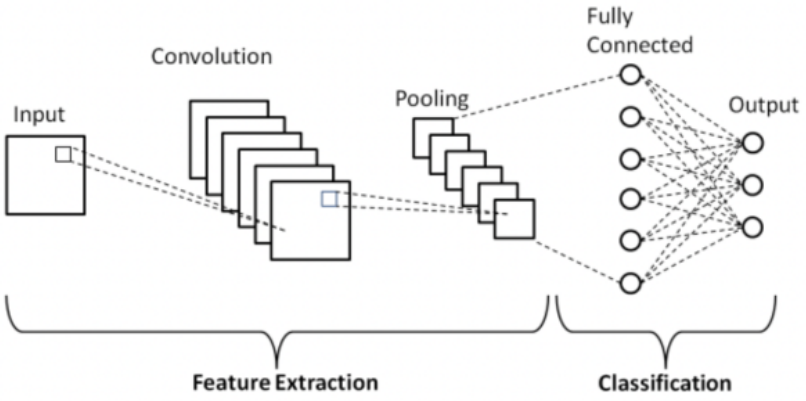

Figure 2.2: Convolutional Neural Networks.[34].

Within the specific context of network traffic analysis and time-series modeling, One-Dimensional Convolutional Neural Networks (1D-CNN) demonstrate exceptional efficacy when deployed to scan across discrete data windows along the temporal axis. The inherent capacity for autonomous hierarchical feature learning from raw data points allows the 1D-CNN to accurately capture critical local variations and transitional states within network packet sequences. Ultimately, this mechanism operates as a robust feature extraction filter, constructing a high-quality, dense representational input space prior to propagating the refined signals to the subsequent sequential learning modules.

2.4 Squeeze-and-Excitation Networks

Squeeze-and-Excitation (SE) Networks are architectures that integrate a channel attention mechanism, designed to enhance the representational power of convolutional neural networks by explicitly modeling the interdependencies between feature channels. In standard CNNs, each output channel is typically evaluated with equal importance. The SE block addresses this limitation by autonomously learning, evaluating, and recalibrating the weight of each distinct channel, allowing the model to flexibly "attend" to the most core features and actively suppress irrelevant, noisy signals [35].

The operational mechanism of the SE block occurs through two primary phases: "Squeeze" and "Excitation". In the Squeeze step, the local spatial information of each channel is aggregated into a global descriptor vector via Global Average Pooling. Subsequently, the Excitation step utilizes a small neural network (typically comprising Fully Connected layers and a Sigmoid activation function) to learn the non-linear interactions among channels, thereby generating a set of continuous weights within the range of [0, 1]. Finally, the original feature maps are directly scaled by these weights, creating a new, highly discriminative, and more optimized feature representation space for the downstream processing modules.

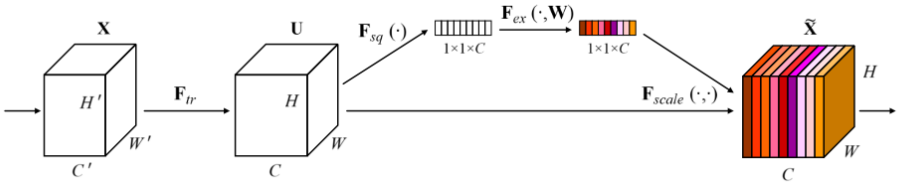

Figure 2.3: Squeeze-and-Excitation block [36].

2.5 Bidirectional Long Short-Term Memory

Bidirectional Long Short-Term Memory (BiLSTM) is an advanced variant of Recurrent Neural Networks (RNN), designed to address the limitations of traditional LSTMs, which can only learn information in a single direction from the past to the present. Architecturally, BiLSTM operates by concurrently integrating two independent LSTM layers: one layer processes the input sequence in the forward direction, while the other processes it in the backward direction. At each time step, the hidden states generated from both layers are combined (via concatenation) to create a comprehensive feature representation, encapsulating both the preceding and succeeding contexts of that specific data point.

The ability to access bidirectional information makes BiLSTM particularly exceptional in time-series analysis or data flow tasks involving complex sequential dependencies. Instead of evaluating a data point based solely on past occurrences, the model can refine its decisions by correlating them with events occurring immediately afterward within the same observation window. In the context of network traffic analysis, this holistic perspective enables the model to more accurately capture targeted attack behaviors or subtle, time-dependent variations in the data.

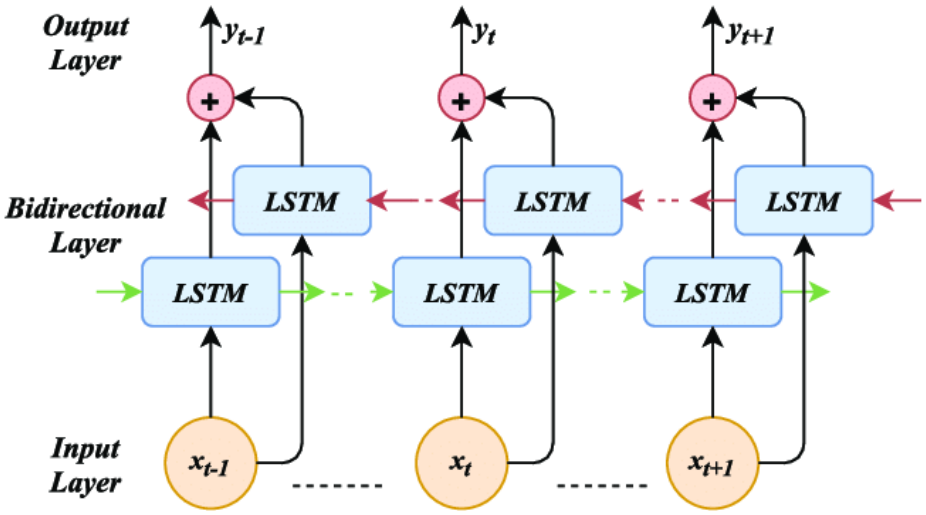

Figure 2.4: BiLSTM [37].

2.6 Graph Attention Network

Graph Attention Networks (GAT) represent a significant advancement in the field of deep learning for graph-structured data. Unlike traditional Graph Convolutional Networks (GCN), which aggregate information from neighboring nodes uniformly or based on a predefined static structure, GAT incorporates a self-attention mechanism directly into the message-passing process between nodes. The core GAT block allows each node to autonomously compute an attention score for each of its neighbors. This weight determines the importance or the amount of information that a neighboring node is permitted to contribute to updating the central node's state representation.

In network intrusion detection tasks, GAT proves highly effective due to its inherent flexibility. It does not require a fixed graph topology and can adeptly process continuously changing structures (dynamic graphs). The ability to learn complex spatial relationships and identify critical "bottlenecks" or "anomalous links" enables GAT to generate exceptionally high-quality embeddings. These embeddings serve as a robust foundation for downstream classification models such as XGBoost.

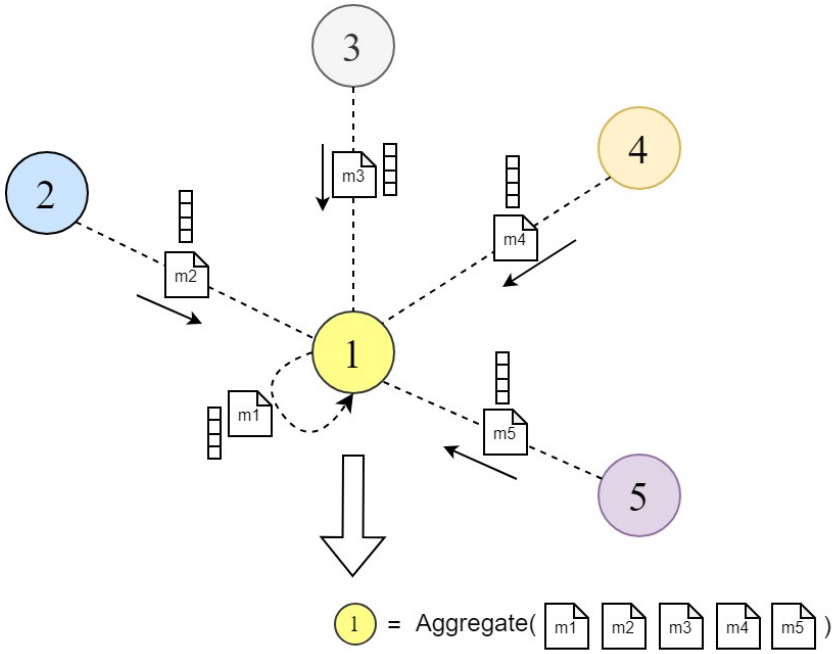

Figure 2.5: GAT mechanism [38].

2.7 XGBoost (eXtreme Gradient Boosting)

XGBoost (eXtreme Gradient Boosting) is a classification and regression machine learning algorithm based on ensemble learning techniques, distinguished by its exceptional computational efficiency and superior accuracy. Unlike methods such as Random Forest, which construct multiple decision trees independently, XGBoost operates on a boosting mechanism. This algorithm trains decision trees sequentially, where each subsequent tree is specifically designed to predict and directly compensate for the residual errors made by the ensemble of preceding trees, achieved by optimizing the loss function via gradient descent.

The core differentiator that establishes XGBoost's superiority over traditional boosting algorithms is the direct integration of regularization mechanisms into the objective function, such as L1 (Lasso) and L2 (Ridge) penalties applied to the leaf weights. This enables the model to autonomously control its complexity and effectively mitigate overfitting. Consequently, XGBoost is frequently deployed as a terminal classifier due to its flexible processing capabilities and its capacity to maximally extract information from complex feature representation spaces.

2.8 Ensemble Learning

Ensemble Learning is an advanced machine learning paradigm in which multiple base learners are trained and combined to solve a specific problem, rather than relying on a single algorithm. By aggregating decisions from multiple models with diverse structural characteristics and hypothesis spaces, an ensemble system can

effectively compensate for local weaknesses, significantly reducing both variance and bias. The result is a classification system exhibiting higher accuracy, robust stability, and superior generalization capabilities on complex datasets.

Among prominent ensemble strategies (such as Bagging and Boosting), the Stacking technique is extensively applied in hybrid detection architectures. Stacking operates by collecting the predictive outputs (typically probability distributions) from independent base models, and subsequently utilizing a higher-level model—a Meta-Learner—to learn how to optimally weight and integrate these outputs. The Meta-Learner mechanism excels at rendering the most comprehensive and highly reliable final classification decision.

CHAPTER 3. METHODOLOGY

3.1 Problem Formulation and Notation

Let a network-flow dataset be denoted by

$$\mathcal{D} = \{(f_i, \mathbf{x}_i, y_i, t_i, s_i, d_i, p_i)\}_{i=1}^{N}, \tag{3.1}$$

where f_i is the i-th flow, $\mathbf{x}_i \in \mathbb{R}^F$ is its normalized feature vector, $y_i \in \{1, \ldots, C\}$ is the class label, t_i is the timestamp, s_i and d_i are the source and destination IP addresses, and p_i contains protocol and port information. The objective is to learn a classifier

$$\Phi : \mathbf{x}_i \mapsto \hat{\mathbf{p}}_i \in [0,1]^C, \qquad \hat{y}_i = \arg \max_{c \in \{1,\ldots,C\}} \hat{p}_{i,c}, \tag{3.2}$$

that assigns each flow to either benign traffic or a specific intrusion class. The proposed method decomposes Φ into two complementary base learners and a meta-learner:

$$\Phi(f_i) = g_{\text{meta}} \left(\hat{\mathbf{p}}_i^{\text{gatxgb}} \parallel \hat{\mathbf{p}}_i^{\text{rcb}} \right), \tag{3.3}$$

where $\parallel$ denotes vector concatenation, $\hat{\mathbf{p}}_i^{\text{gatxgb}}$ is produced by the spatial GAT–XGBoost branch, and $\hat{\mathbf{p}}_i^{\text{rcb}}$ is produced by the temporal Residual CNN–BiLSTM branch.

3.2 Overall Architecture

The proposed framework contains two base learners and one meta-learner. The spatial branch constructs a flow-based graph and learns topology-aware embeddings using graph attention. These embeddings, optionally concatenated with raw normalized features, are classified by XGBoost. The temporal branch converts ordered flows into sliding windows, extracts local temporal patterns by residual convolutional blocks, and models long-range dependencies by BiLSTM attention. The meta-learner fuses the predictive distributions from both branches.

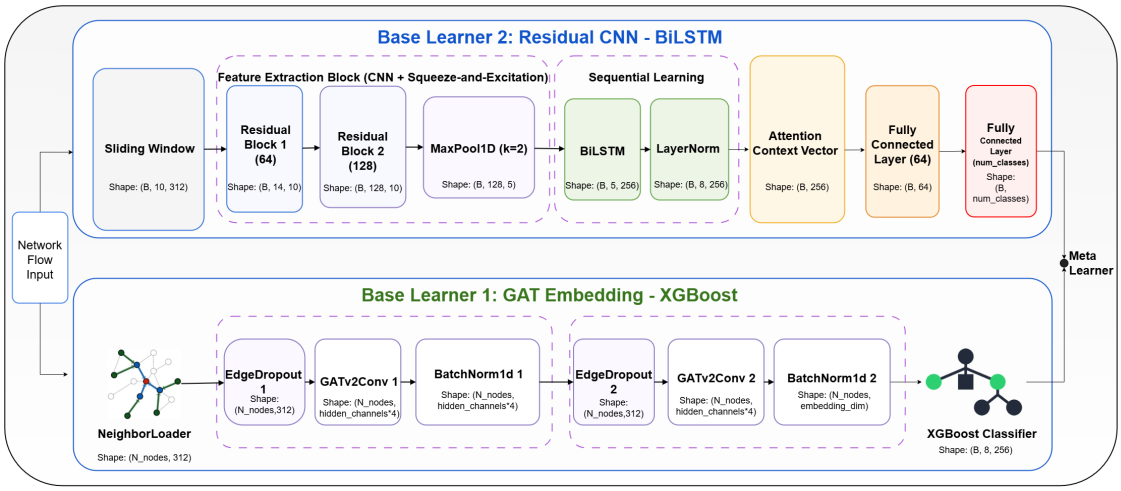

Figure 3.1: Overall architecture of the proposed ensemble intrusion detection framework.

3.3 Flow-Based Graph Construction

Instead of representing hosts as nodes and flows as edges, the thesis uses a flow-as-node graph. This design is motivated by dense IoT traffic, where a small number of hosts may exchange millions of flows. Compressing such flows into host-to-host edges can create information bottlenecks and can also make graph aggregation vulnerable to oversmoothing [39], [40]. In the proposed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, each node $v_i \in \mathcal{V}$ corresponds to one flow f_i, and its node feature is $\mathbf{x}_i$.

For each flow f_i, directed candidate edges are only created from historical flows to preserve causality and prevent temporal leakage. A directed edge (j, i) is added when

$$0 < t_i - t_j \leq \tau_{\max}, \qquad \mathbb{I}(s_i = s_j \vee d_i = d_j) = 1, \tag{3.4}$$

and only the K nearest historical neighbors are kept:

$$\mathcal{N}_K(i) = \mathrm{TopK}_j \left(-|t_i - t_j|\right), \qquad K = 10. \tag{3.5}$$

Each edge is associated with a structural feature vector

$$\mathbf{e}_{ji} = \left[\Delta t_{ji}, \mathbb{I}(s_i = s_j \wedge d_i = d_j), \mathbb{I}(\mathrm{sport}_i = \mathrm{sport}_j), \mathbb{I}(\mathrm{dport}_i = \mathrm{dport}_j)\right], \tag{3.6}$$

where $\Delta t_{ji} = t_i - t_j$. Algorithm 3.1 summarizes the construction procedure.

Algorithm 3.1: Pseudo-code for flow-based graph construction.

Input: Chronologically unordered flows $\mathcal{F}$, time threshold $\tau_{\max}$, maximum degree K.

Output: Directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with edge features.

Sort all flows by timestamp so that $t_1 \leq t_2 \leq \cdots \leq t_N$;

Create one node v_i for each flow f_i and assign node feature $\mathbf{x}_i$;

for *each current flow* f_i **do**

- Scan only historical candidates f_j with $j < i$;
- Keep candidate f_j if $0 < t_i - t_j \leq \tau_{\max}$ and the two flows share source or destination IP;
- Rank candidates by increasing $|t_i - t_j|$ and retain at most K neighbors;
- Add directed edge (j, i) for each retained neighbor and compute $\mathbf{e}_{ji}$;

return $\mathcal{G}$ *for GAT embedding learning*;

3.4 Base Learner 1: GAT Embedding - XGBoost

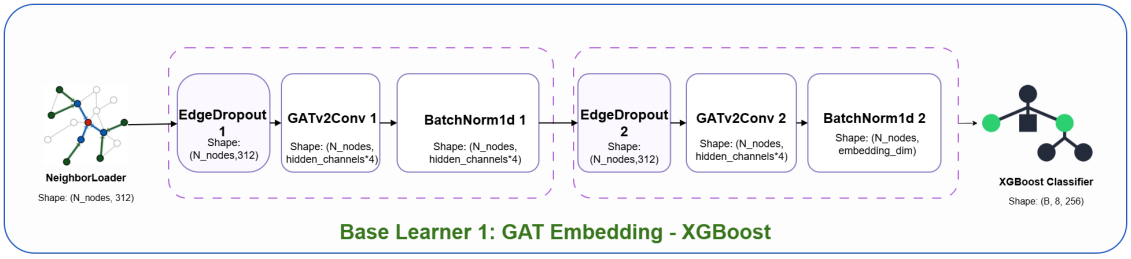

Figure 3.2: Spatial base learner based on GAT embeddings and XGBoost classification.

Given the constructed graph, the GAT branch computes an embedding $\mathbf{h}_i$ for each flow node. The initial hidden state is

$$\mathbf{h}_i^{(0)} = \mathbf{W}_0 \mathbf{x}_i + \mathbf{b}_0. \tag{3.7}$$

Following the original Graph Attention Network formulation [1], for a GATv2-style attention layer [41], the unnormalized attention score from neighbor j to node i is

$$e_{ij}^{(m)} = \mathbf{a}_m^{\top} \mathrm{LeakyReLU}\left(\mathbf{W}_m [\mathbf{h}_i^{(\ell)} \parallel \mathbf{h}_j^{(\ell)} \parallel \mathbf{e}_{ji}]\right), \tag{3.8}$$

where m indexes the attention head. The normalized attention coefficient is

$$\alpha_{ij}^{(m)} = \frac{\exp(e_{ij}^{(m)})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik}^{(m)})}. \tag{3.9}$$

The node representation is updated by aggregating the neighbor messages:

$$\mathbf{h}_i^{(\ell+1)} = \Big\Vert_{m=1}^{M} \sigma\left(\sum_{j\in\mathcal{N}(i)} \alpha_{ij}^{(m)} \mathbf{W}_m \mathbf{h}_j^{(\ell)}\right), \tag{3.10}$$

where M is the number of attention heads and $\sigma(\cdot)$ is a nonlinear activation function. After the final graph layer, the learned embedding is denoted by $\mathbf{z}_i = \mathbf{h}_i^{(L)}$.

Two spatial variants are evaluated. The embedding-only variant uses $\mathbf{z}_i$ as input to XGBoost:

$$\hat{\mathbf{p}}_i^{\text{gatxgb}} = \text{XGBoost}(\mathbf{z}_i). \tag{3.11}$$

The best-performing variant concatenates graph embeddings with raw normalized features:

$$\tilde{\mathbf{x}}_i = \mathbf{z}_i \parallel \mathbf{x}_i, \qquad \hat{\mathbf{p}}_i^{\text{gatrawxgb}} = \text{XGBoost}(\tilde{\mathbf{x}}_i). \tag{3.12}$$

This design lets the branch exploit both topology-aware representations and original tabular traffic attributes. For the multi-class XGBoost classifier, the additive tree model is written as

$$\hat{\mathbf{p}}_i = \text{Softmax}\left(\sum_{q=1}^{Q} \mathbf{f}_q(\tilde{\mathbf{x}}_i)\right), \qquad \mathbf{f}_q \in \mathcal{F}_{\text{tree}}, \tag{3.13}$$

and the regularized training objective is

$$\mathcal{L}_{\text{xgb}} = \sum_{i=1}^{N} \ell(y_i, \hat{\mathbf{p}}_i) + \sum_{q=1}^{Q} \Omega(\mathbf{f}_q), \qquad \Omega(\mathbf{f}) = \gamma T_{\mathbf{f}} + \frac{1}{2}\lambda \|\mathbf{w}_{\mathbf{f}}\|_2^2, \tag{3.14}$$

where $T_{\mathbf{f}}$ is the number of leaves and $\mathbf{w}_{\mathbf{f}}$ contains leaf weights. This objective is useful for tabular IDS data because it balances fitting capacity with explicit complexity control.

3.5 Base Learner 2: Residual CNN–BiLSTM

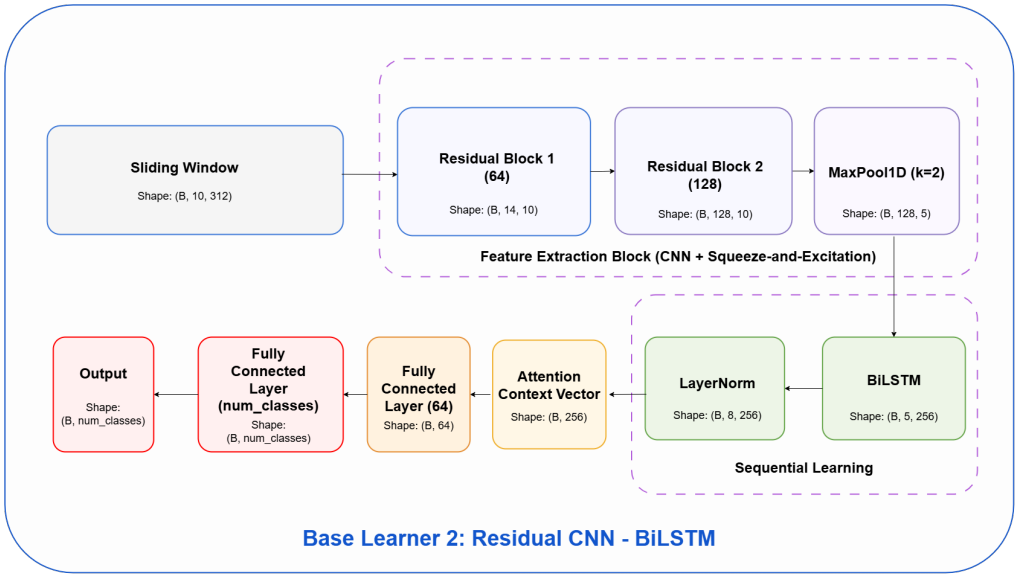

Figure 3.3: Temporal base learner based on residual CNN blocks and BiLSTM attention.

The temporal branch starts from chronologically ordered flows and constructs fixed-length windows of size T:

$$\mathbf{S}_i = [\mathbf{x}_{i-T+1}, \mathbf{x}_{i-T+2}, \ldots, \mathbf{x}_i] \in \mathbb{R}^{T \times F}. \tag{3.15}$$

Each window is assigned the label of its last flow, which makes the prediction causal with respect to the current flow. An one-dimensional convolution extracts local temporal features:

$$\mathbf{U} = \mathrm{Conv1D}(\mathbf{S}_i; \mathbf{W}_{\mathrm{conv}}) + \mathbf{b}_{\mathrm{conv}}. \tag{3.16}$$

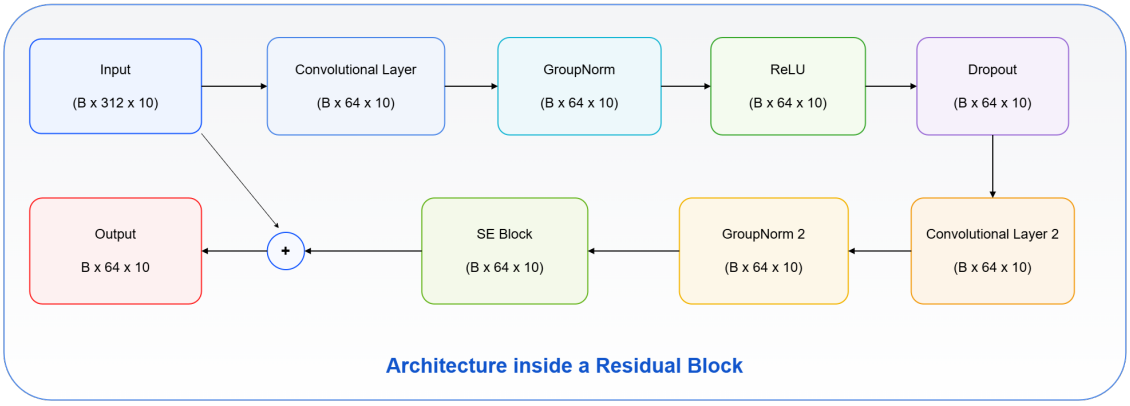

Figure 3.4: Residual block used in the temporal base learner.

Figure 3.4 illustrates the internal structure of the residual block before the block

is formalized mathematically. A residual block is defined as

$$\mathbf{R} = \mathrm{ReLU}\left(\mathcal{F}_{\mathrm{res}}(\mathbf{U}) + \mathcal{P}(\mathbf{U})\right), \tag{3.17}$$

where $\mathcal{F}_{\mathrm{res}}(\cdot)$ is a stack of convolution, normalization, and activation operations, and $\mathcal{P}(\cdot)$ is either the identity mapping or a projection used to match channel dimensions. Each residual block also contains a Squeeze-and-Excitation recalibration unit [35], [42]. Given an intermediate feature map $\mathbf{U} \in \mathbb{R}^{C\times T}$, channel-wise squeeze statistics are computed by

$$z_c = \frac{1}{T}\sum_{t=1}^{T} U_{c,t}, \qquad c = 1, \ldots, C. \tag{3.18}$$

The excitation vector is then generated by

$$\mathbf{s} = \sigma\left(\mathbf{W}_2\, \delta(\mathbf{W}_1 \mathbf{z})\right), \tag{3.19}$$

where $\delta(\cdot)$ is ReLU and $\sigma(\cdot)$ is the sigmoid function. The recalibrated output is

$$\tilde{U}_{c,t} = s_c\, U_{c,t}. \tag{3.20}$$

This mechanism adaptively emphasizes informative temporal channels and suppresses noisy channels. The two residual blocks used in this branch are configured as summarized in Table 3.1.

Table 3.1: Configuration of residual blocks in the temporal base learner.

Parameter	Residual Block 1	Residual Block 2
Input channels	F	64
Output channels	64	128
Kernel size	3	3
Padding	1	1
Group normalization	8 groups	8 groups
SE reduction ratio	8	8
Dropout rate	0.2	0.2
Output shape	$B \times 64 \times T$	$B \times 128 \times T$

After temporal pooling, the CNN representation is transposed into a sequence

$\{\bar{\mathbf{x}}_t\}_{t=1}^{T'}$ and processed by a bidirectional LSTM:

$$\overrightarrow{\mathbf{h}}_t = \mathrm{LSTM}_{\mathrm{fw}}(\bar{\mathbf{x}}_t, \overrightarrow{\mathbf{h}}_{t-1}), \tag{3.21}$$

$$\overleftarrow{\mathbf{h}}_t = \mathrm{LSTM}_{\mathrm{bw}}(\bar{\mathbf{x}}_t, \overleftarrow{\mathbf{h}}_{t+1}), \tag{3.22}$$

$$\mathbf{h}_t = \overrightarrow{\mathbf{h}}_t \parallel \overleftarrow{\mathbf{h}}_t. \tag{3.23}$$

Attention scores and weights are computed as

$$e_t = \mathbf{v}_{\mathrm{att}}^{\top} \tanh(\mathbf{W}_{\mathrm{att}}\mathbf{h}_t + \mathbf{b}_{\mathrm{att}}), \qquad \alpha_t = \frac{\exp(e_t)}{\sum_{j=1}^{T'} \exp(e_j)}. \tag{3.24}$$

The final temporal context vector is

$$\mathbf{c} = \sum_{t=1}^{T'} \alpha_t \mathbf{h}_t. \tag{3.25}$$

The detailed BiLSTM-attention configuration used after the residual blocks is summarized in Table 3.2.

Table 3.2: Configuration of the BiLSTM-attention module.

Parameter	Value
Input tensor shape	$B \times T' \times 128$ ($T' = 5$)
Hidden size	128
BiLSTM output dimension	256 (128×2)
Attention activation	Tanh
Normalization layer	LayerNorm with 256 units
Context vector shape	$B \times 256$

The branch prediction is generated by a fully connected classifier with dropout and layer normalization:

$$\hat{\mathbf{p}}_i^{\mathrm{rcb}} = \mathrm{Softmax}\left(\mathbf{W}_2\,\mathrm{Dropout}\left(\mathrm{ReLU}(\mathrm{LayerNorm}(\mathbf{W}_1\mathbf{c} + \mathbf{b}_1))\right) + \mathbf{b}_2\right). \tag{3.26}$$

3.6 Meta-Learner

The meta-learner receives the class-probability vectors from the two base learners:

$$\mathbf{m}_i = \hat{\mathbf{p}}_i^{\mathrm{gatrawxgb}} \parallel \hat{\mathbf{p}}_i^{\mathrm{rcb}} \in \mathbb{R}^{2C}. \tag{3.27}$$

Two meta-classifier variants are evaluated. The logistic-regression meta-learner computes

$$\hat{\mathbf{p}}_i^{\text{final}} = \text{Softmax}(\mathbf{W}_{\text{LR}}\mathbf{m}_i + \mathbf{b}_{\text{LR}}), \tag{3.28}$$

whereas the XGBoost meta-learner uses boosted decision trees [2]:

$$\hat{\mathbf{p}}_i^{\text{final}} = \text{XGBoost}_{\text{meta}}(\mathbf{m}_i). \tag{3.29}$$

To prevent information leakage, the stacked learning protocol is separated across the train, validation, and test splits. First, both base learners are trained on the training set. Second, the trained base learners generate class-probability vectors on the validation set, and these validation probabilities are concatenated to form the meta-training matrix. The meta-learner is trained only on this validation-level representation. Finally, during testing, the same trained base learners generate probability vectors on the independent test set, and these test probabilities are passed to the trained meta-learner to produce the final predictions.

The final predicted label is obtained as $\hat{y}_i = \arg\max_c \hat{p}_{i,c}^{\text{final}}$. In this formulation, $\hat{y}_i$ represents the intrusion category assigned to the current network flow, including either the benign class or one of the predefined attack classes in the corresponding use case. Therefore, the output of the proposed architecture is not only a confidence distribution over all classes, but also a concrete decision that can be used by downstream monitoring or alerting components. This completes the end-to-end mapping from raw flow features to actionable results.

4.1 Evaluation Dataset

Network intrusion detection research has commonly relied on public benchmarks such as UNSW-NB15, CICIDS2017, Bot-IoT, and TON-IoT [23], [24], [25], [26]. To validate the proposed model's efficacy against the latest paradigms in Internet of Things (IoT) cyberattacks, the study utilizes two recently published benchmark datasets explicitly designed for **Network Intrusion Detection Systems (NIDS)**: the **BCCC-IoT-IDS-Zwave-2025** [3] and **CIC IIoT 2025 datasets** [4].

4.1.1 BCCC-IoT-IDS-Zwave-2025 Dataset

The BCCC-IoT-IDS-Zwave-2025 dataset is a large-scale, specialized cybersecurity benchmark developed to facilitate advanced research in intrusion detection and cyberattacks targeting Smart Home and Internet of Things (IoT) ecosystems. This comprehensive dataset includes five diverse data sources: IoT-Zwave communication signals, IP-based network traffic, device-level telemetry, MQTT message traffic, and system logs [3].

Designed specifically for the task of network anomaly detection and flow classification, this study exclusively leverages the IP-based network traffic data provided in tabular CSV format. The generation of these CSV files was executed utilizing the open-source network traffic analyzer NTLFlowLyzer, which systematically converts raw packet captures (PCAP) into structured network flows predicated on standard 5-tuple attributes. A key methodological advancement of this dataset is its multi-time-window flow extraction strategy, utilizing intervals of 10, 30, 300, and 3600 seconds. This multi-granular approach offers a multidimensional analytical perspective: narrow time windows (10s and 30s) empower the model to detect rapid, burst-oriented attacks, whereas wider temporal windows (300s and 3600s) supply the extended contextual baseline required to identify stealthy, slow-rate resource exhaustion anomalies.

In terms of the threat landscape, the dataset features a robust diversity of resource exhaustion attack scenarios specifically targeting two critical network layers: the Transport Layer and the Application Layer. Each data record includes a set of refined, high-resolution statistical features, including packet size distributions, Inter-Arrival Times (IAT), TCP flag ratios, and connection handshake states. To align with practical deployment requirements and network defense strategies, the proposed architecture is trained and evaluated to detect attacks at these two layers independently. Consequently, the empirical evaluation is partitioned into two distinct

experimental scenarios: **Use Case 1: HTTP-based Attack - IoT-IDS-Zwave-2025** and **Use Case 2: Application-based Attack - IoT-IDS-Zwave-2025**.

a, Use Case 1: HTTP-Based Attacks - IoT-IDS-Zwave-2025

The IoT-IDS-Zwave-2025 dataset encompasses **11** distinct Distributed Denial of Service (DDoS) attack variants specifically targeting the **HTTP layer** of the network infrastructure. For this particular experimental use case, the total volume of extracted network flows amounts to a substantial **3,801,012** samples. The detailed quantitative distribution of these samples across each specific attack category is illustrated as follows:

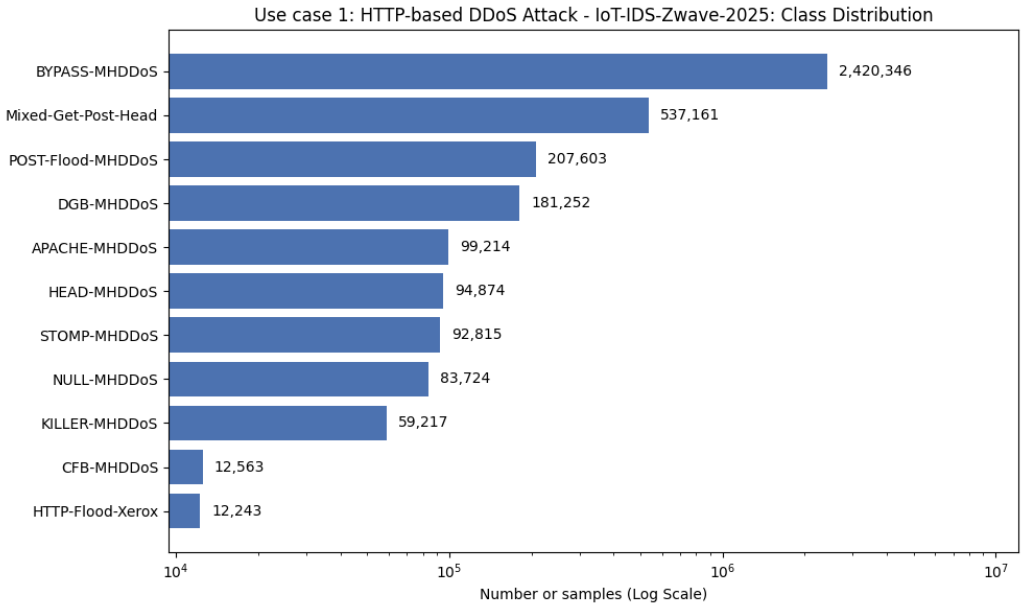

Figure 4.1: HTTP-Based Attacks - IoT-IDS-Zwave-2025: Class Distribution.

b, Use Case 2: Application-based Attack - IoT-IDS-Zwave-2025

Shifting the focus to the **Application Layer** of the network architecture, the dataset comprises **13** diverse Distributed Denial of Service (DDoS) attack variants. For this specific experimental use case, the total volume of extracted network flows aggregates to an extensive **10,946,824** samples. The precise quantitative distribution of these flows across each respective attack class is detailed in the following chart:

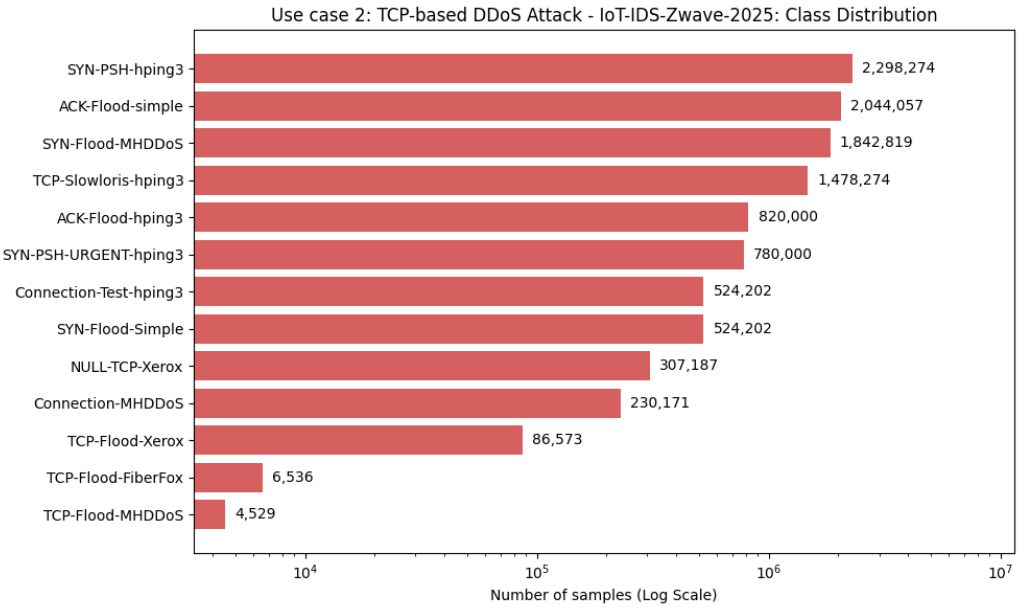

Figure 4.2: Application-Based Attacks - IoT-IDS-Zwave-2025: Class Distribution.

4.1.2 CIC IIoT 2025 Dataset

The CIC IIoT 2025 dataset is a novel and comprehensive benchmark released by the Canadian Institute for Cybersecurity (CIC) [4]. It is explicitly engineered to facilitate the research and development of robust Intrusion Detection Systems (IDS) tailored for Industrial Internet of Things (IIoT) environments. Closely simulating the complexities of real-world network conditions, the dataset exhibits severe multi-class imbalance, with flow-based samples systematically categorized into seven primary macro-groups.

A distinguishing architectural feature of this dataset is its temporal aggregation methodology. Rather than being captured at a single fixed timestamp, the network flows are extracted and aggregated across four distinct time windows: 1 second, 2 seconds, 5 seconds, and 10 seconds. Aligning with the temporal processing characteristics of the proposed model and the core objectives of this research, the experimental analysis is **exclusively focused on the network flows aggregated within the 1-second time window.**

For this specific evaluation benchmark, the fundamental classification objective is to accurately categorize each network flow into **one of the 7 attack categories or as benign traffic**. Within the selected 1-second aggregation window, the dataset comprises a total of **227,191** samples. The precise quantitative distribution of these samples across the respective classes is illustrated in the following chart:

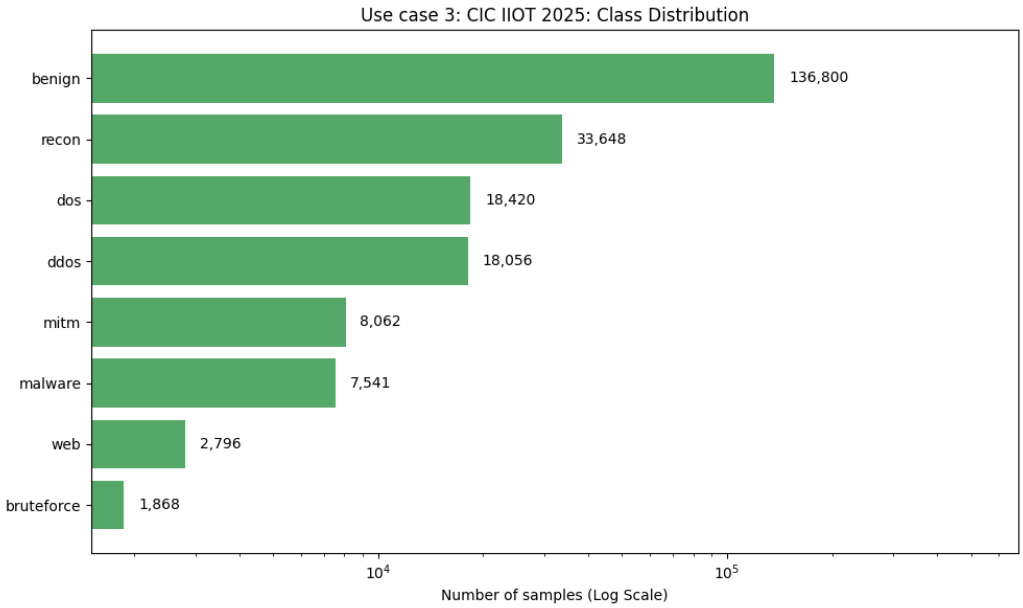

Figure 4.3: CIC IIoT 2025: Class Distribution.

4.2 Data Preprocessing

The selected datasets are subjected to a rigorous and standardized preprocessing pipeline. This workflow encompasses fundamental procedures including missing value imputation, one-hot encoding for categorical features, and label encoding for the target classes. Furthermore, to guarantee numerical stability and facilitate optimal gradient convergence for the Deep Learning architectures, a QuantileTransformer is applied for robust feature normalization. To ensure a fair and unbiased comparative analysis, all baseline and proposed experimental models are trained and evaluated utilizing this identically preprocessed data pipeline.

Regarding the dataset partitioning into training, validation, and testing subsets, a strict chronological ordering is initially enforced across all datasets to preclude temporal data leakage. Subsequently, to maintain consistent class distributions across all subsets, a hybrid stratified and time-based splitting methodology is implemented. Specifically, the data is first stratified by class label; within each individual class stratum, a strict chronological split is then applied. This dual approach preserves the temporal continuity of the network flows while maintaining class balance.

Through iterative experimental testing, and to deliberately preserve the phenomenon of **Concept Drift** inherent in large-scale cyberattacks, the datasets for each experimental use case are partitioned into training, validation, and testing sets according to the following strategic ratios:

- **Use Case 1 (HTTP-based Attack - IoT-IDS-Zwave-2025):** 65:15:20

- **Use Case 2 (Application-based Attack - IoT-IDS-Zwave-2025):** 30:40:30
- **Use Case 3 (CIC IIoT 2025):** 70:15:15

4.3 Evaluation Metrics

To provide a comprehensive and fair assessment of the proposed intrusion detection framework, this thesis evaluates all models using four complementary metrics: **Macro Precision, Macro Recall, Macro F1, and AUC-ROC.** Because the evaluation benchmarks are multi-class and highly imbalanced, relying solely on overall accuracy would be insufficient: a model may achieve a high accuracy by favoring majority classes while still failing to recognize rare but security-critical attack categories. Therefore, the evaluation protocol emphasizes macro-averaged metrics, which compute performance independently for each class and then average the results across all classes. This gives equal importance to benign traffic and every attack category, making the metrics more suitable for assessing robustness in realistic NIDS scenarios.

For a class c, let TP_c, FP_c, and FN_c denote the number of true positives, false positives, and false negatives, respectively. The class-wise precision measures how many samples predicted as class c are actually correct:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}. \tag{4.1}$$

The class-wise recall measures how many true samples of class c are successfully detected:

$$\text{Recall}_c = \frac{TP_c}{TP_c + FN_c}. \tag{4.2}$$

Given C total classes, Macro Precision and Macro Recall are computed by averaging the corresponding class-wise values:

$$\text{Macro Precision} = \frac{1}{C}\sum_{c=1}^{C} \text{Precision}_c, \qquad \text{Macro Recall} = \frac{1}{C}\sum_{c=1}^{C} \text{Recall}_c. \tag{4.3}$$

Macro Precision reflects the reliability of the model's predicted labels across all classes, while **Macro Recall** reflects its ability to cover and detect each class, including minority attacks. In the context of intrusion detection, recall is particularly important because missing an attack instance may lead to an undetected security incident. However, precision is also necessary to limit excessive false alarms, which can reduce the practical usability of an IDS in real monitoring environments.

Macro F1 is used as the primary balanced metric because it combines precision

and recall through their harmonic mean. For each class, the F1-score is defined as

$$\mathrm{F1}_c = \frac{2 \cdot \mathrm{Precision}_c \cdot \mathrm{Recall}_c}{\mathrm{Precision}_c + \mathrm{Recall}_c}, \tag{4.4}$$

and the macro-averaged F1-score is calculated as

$$\text{Macro F1} = \frac{1}{C} \sum_{c=1}^{C} \mathrm{F1}_c. \tag{4.5}$$

This metric is especially useful when the class distribution is skewed, since it penalizes models that perform well on common classes but poorly on minority classes. As a result, Macro F1 is used throughout the experimental analysis to compare baseline models, proposed base learners, and meta-learner ensembles under a unified criterion.

In addition to threshold-dependent classification metrics, **AUC-ROC** is reported to evaluate the ranking quality of probabilistic predictions. The ROC curve describes the trade-off between the true positive rate and the false positive rate at different classification thresholds. For multi-class intrusion detection, this thesis adopts a one-vs-rest interpretation, where each class is evaluated against all remaining classes and the resulting values are averaged. A higher AUC-ROC indicates that the model assigns higher confidence to correct classes over incorrect alternatives across a broad range of thresholds. This complements Macro F1 by showing whether a model produces well-separated probability scores, even when the final predicted labels are affected by class imbalance or threshold selection.

4.4 Experimental Environment and Setup

All empirical experiments, system testing, and model evaluations were conducted on a personal computer configured for deep learning and large-scale data processing. The hardware infrastructure consists of an Intel Core i5-14600K processor (14 cores, 20 threads, up to 5.3 GHz), 32 GB DDR5 RAM (6000 MHz), an NVIDIA GeForce RTX 4070 SUPER GPU with 12 GB VRAM, and a 1 TB NVMe Gen 4.0 SSD.

The software environment was implemented in PyTorch, with GPU acceleration provided by CUDA 12.8 and compatible cuDNN drivers. To ensure statistical robustness and mitigate training variance, **each standalone base model was independently trained and evaluated three times**, and the final performance metrics are reported as the arithmetic mean of these iterations. For the proposed Meta-Learner ensemble architecture, **the overarching model integrates the two best-performing base learners selected from the prior independent training**

runs, thereby ensuring optimal predictive efficacy.

CHAPTER 5. ABLATION STUDY

To rigorously validate the architectural efficiency and optimize the predictive capabilities of the proposed intrusion detection system, a comprehensive ablation study is conducted across both the foundational base learners and the overarching meta-learner.

The experimental training and evaluation protocols are strategically differentiated based on the model type:

- **Base Learners Protocol:** For the standalone base models, the experimental ablation configurations are trained strictly on the designated training set. Subsequently, hyperparameter tuning and model selection are performed using the validation set, before the final performance is benchmarked on an isolated, independent testing set to ensure unbiased metrics.
- **Meta-Learner Protocol:** To avoid information leakage in the stacking stage, the base learners are first trained only on the training set. Their predictive probability vectors are then generated on the validation set and used as the training features for the meta-learner. During final evaluation, the trained base learners produce probability vectors on the isolated testing set, and these test probabilities are passed to the trained meta-learner to obtain the final predictions.

5.1 Ablation Study for the Residual CNN-BiLSTM Base Learner

Table 5.1: Ablation Study Results for Residual CNN-BiLSTM Variations across Three Use Cases

Variations	Macro Precision	Macro Recall	Macro F1
Use Case 1			
Proposed (US+RB+SW+Base+Att)	**0.8695 ± 0.0160**	**0.9184 ± 0.0033**	**0.8794 ± 0.0138**
w/o Att (US+RB+SW+Base)	0.8352 ± 0.0117	0.8936 ± 0.0083	0.8416 ± 0.0152
w/o SW (US+RB+Base+Att)	0.7648 ± 0.0371	0.7991 ± 0.0422	0.7624 ± 0.0455
w/o RB (US+SW+Base+Att)	0.8371 ± 0.0101	0.8911 ± 0.0119	0.8425 ± 0.0144
w/o RB, SW (US+Base+Att) [11]	0.7584 ± 0.0328	0.7865 ± 0.0266	0.7505 ± 0.0394
w/o RB, SW (HS+Base+Att)	0.7921 ± 0.0134	0.8582 ± 0.0003	0.8044 ± 0.0072
Use Case 2			
Proposed (US+RB+SW+Base+Att)	**0.9218 ± 0.0151**	**0.9581 ± 0.0021**	**0.9333 ± 0.0105**
w/o Att (US+RB+SW+Base)	0.8805 ± 0.0517	0.9153 ± 0.0350	0.8886 ± 0.0429
w/o SW (US+RB+Base+Att)	0.9209 ± 0.0136	0.9392 ± 0.0117	0.9197 ± 0.0151
w/o RB (US+SW+Base+Att)	0.8277 ± 0.0047	0.8869 ± 0.0029	0.8411 ± 0.0040
w/o RB, SW (US+Base+Att) [11]	0.9040 ± 0.0096	0.9401 ± 0.0042	0.9110 ± 0.0062
w/o RB, SW (HS+Base+Att)	-	-	-
Use Case 3			
Proposed (US+RB+SW+Base+Att)	**0.7593 ± 0.0081**	**0.7459 ± 0.0321**	**0.7182 ± 0.0116**
w/o Att (US+RB+SW+Base)	0.7011 ± 0.0097	0.6670 ± 0.0060	0.6542 ± 0.0025
w/o SW (US+RB+Base+Att)	0.6234 ± 0.0188	0.5838 ± 0.0057	0.5907 ± 0.0117
w/o RB (US+SW+Base+Att)	0.7469 ± 0.0202	0.7113 ± 0.0486	0.6971 ± 0.0335
w/o RB, SW (US+Base+Att) [11]	0.5998 ± 0.0127	0.5852 ± 0.0053	0.5792 ± 0.0072
w/o RB, SW (HS+Base+Att)	0.6480 ± 0.0248	0.6250 ± 0.0109	0.6172 ± 0.0077

Note: **Base**: CNN + LSTM; **US**: UnderSampling; **HS**: Hybrid Sampling (SMOTE + UnderSampling); **RB**: Residual Block; **SW**: Sliding Window; **Att**: Attention; **w/o**: Without.

The results of the ablation study across the three evaluated use cases demonstrate that the complete, proposed Residual CNN-BiLSTM base learner achieves the most comprehensive and stable performance, recording Macro F1 scores of 87.94%, 93.33%, and 71.82%, respectively. The systematic isolation of individual components explicitly validates their architectural contributions. Specifically, eliminating the Attention mechanism or substituting the Residual Blocks with standard CNN layers consistently diminishes accuracy metrics, proving that advanced feature refinement and temporal sequence weighting are indispensable for capturing complex attack patterns.

Most notably, the absence of the Sliding Window technique precipitates a severe

performance degradation across all three scenarios—exemplified in Use Case 3, where the Macro F1 score drastically decreases from 71.82% to 59.07%. This substantial decline affirms the critical role of the sliding window in maintaining a strict and continuous temporal context. Ultimately, the integration of UnderSampling, Residual Blocks, Sliding Windows, and the LSTM-Attention module constitutes the optimal architectural configuration, ensuring maximum predictive reliability and robust resilience against data noise.

5.2 Ablation Study for the GAT Embedding + XGBoost Base Learner

Table 5.2: Ablation Study Results for GAT and XGBoost Architectures across Three Use Cases

Model	Macro Precision	Macro Recall	Macro F1
Use Case 1			
GAT Embedding + Raw Features + XGBoost (Proposed)	**0.9333 ± 0.0018**	0.9162 ± 0.0040	**0.9205 ± 0.0020**
GAT Embedding + XGBoost (Proposed)	0.9287 ± 0.0085	**0.9210 ± 0.0083**	0.9202 ± 0.0085
GAT Only	0.8797 ± 0.0164	**0.9210 ± 0.0053**	0.8870 ± 0.0197
XGBoost Only	0.8445 ± 0.0000	0.8788 ± 0.0000	0.8501 ± 0.0000
Use Case 2			
GAT Embedding + Raw Features + XGBoost (Proposed)	0.9504 ± 0.0058	**0.9682 ± 0.0033**	0.9544 ± 0.0057
GAT Embedding + XGBoost (Proposed)	0.9385 ± 0.0146	0.9502 ± 0.0224	0.9369 ± 0.0229
GAT Only	0.9497 ± 0.0009	**0.9682 ± 0.0017**	0.9548 ± 0.0007
XGBoost Only	**0.9531 ± 0.0000**	0.9668 ± 0.0000	**0.9568 ± 0.0000**
Use Case 3			
GAT Embedding + Raw Features + XGBoost (Proposed)	0.7439 ± 0.0080	0.6531 ± 0.0042	0.6809 ± 0.0047
GAT Embedding + XGBoost (Proposed)	0.7439 ± 0.0076	**0.6540 ± 0.0060**	**0.6835 ± 0.0065**
GAT Only	**0.7463 ± 0.0029**	0.6382 ± 0.0057	0.6661 ± 0.0060
XGBoost Only	0.6678 ± 0.0059	0.5993 ± 0.0018	0.6116 ± 0.0043

The results of the ablation study across all three use cases demonstrate that integrating Graph Attention Networks (GAT) with the Extreme Gradient Boosting (XGBoost) algorithm yields substantial performance improvements compared to deploying either model in isolation. Specifically, when utilizing the standard XGBoost or GAT independently, the Macro F1 scores in Use Case 1 and Use Case 3 remain relatively modest (with standalone XGBoost achieving only 85.01% and 61.16%, respectively). However, under the proposed hybrid architecture (GAT Embedding + XGBoost), the classification capability surges to 92.02% in Use Case 1 and 68.35% in Use Case 3. This significant enhancement substantiates that the extracted spatial embedding vectors provide invaluable topological information, thereby significantly optimizing the efficacy of the tree-based classifier.

Notably, upon comparing the two proposed structural variants, the architecture that incorporates original input features (GAT Embedding + Raw Features

+ XGBoost) exhibits the most comprehensive stability. This specific variant not only sustains an excellent F1 score in Use Case 1 (92.05%) but also successfully mitigates the performance degradation observed in Use Case 2—achieving 95.44%, compared to the 93.69% recorded by the variant lacking raw features. This confirms that concurrently preserving raw attributes alongside graph embeddings prevents the loss of crucial local information. Ultimately, this dual-feature approach constructs a richer, multi-dimensional, and highly reliable representation space for the XGBoost classifier.

5.3 Ablation Study for the Meta-Learner Architecture

Table 5.3: Ablation Study Results for Meta-Learner Architectures across Two Use Cases

Model	Macro Precision	Macro Recall	Macro F1
Use Case 1			
Proposed Meta-Learner (LR)	0.9630	0.9654	0.9638
Proposed Meta-Learner (XGBoost)	0.9705	**0.9702**	**0.9697**
Base: Residual CNN-BiLSTM	0.8985	0.9324	0.9028
Base: GAT + XGBoost	**0.9706**	0.9659	0.9672
Use Case 2			
Proposed Meta-Learner (LR)	**0.9554**	0.9665	**0.9571**
Proposed Meta-Learner (XGBoost)	0.9456	**0.9700**	0.9527
Base: Residual CNN-BiLSTM	0.9454	0.9647	0.9471
Base: GAT + XGBoost	0.9502	0.9620	0.9523

Note: **Proposed Meta-Learner** combines spatial features (GAT Embedding + Raw + XGBoost) and temporal features (Residual CNN-BiLSTM). **LR**: Logistic Regression meta-classifier; **XGBoost**: XGBoost meta-classifier; **Base**: Independent base learners before ensemble.

The empirical results evaluating the Meta-Learner framework against the independent Base Learners indicate that ensemble fusion can improve classification performance in some settings, but the improvement is not yet consistently superior to the strongest base learner. When deployed in isolation, the temporal-focused model (Residual CNN-BiLSTM) achieves F1-Scores of 90.28% in Use Case 1 and 94.71% in Use Case 2. Meanwhile, the spatially specialized GAT Embedding + XGBoost learner already provides highly competitive results, recording F1-Scores of 96.72% and 95.23%, respectively.

After the predictive probabilities from both foundational models are fused by the Meta-Learner, the overall performance shows only moderate gains. Specifically, the XGBoost-based Meta-Learner reaches the best F1-Score in Use Case 1 at 96.97%, slightly exceeding the GAT + XGBoost base learner. In Use Case 2, the

Logistic Regression Meta-Learner obtains the highest F1-Score of 95.71%, while the XGBoost Meta-Learner remains close to, but does not clearly surpass, the GAT + XGBoost base learner. These results suggest that the current Meta-Learner architecture is beneficial, yet its advantage over the strongest spatial base learner remains marginal rather than decisive.

Therefore, the Meta-Learner should be viewed as a promising but still improvable component of the proposed system. Although it can exploit complementary spatial and temporal predictive signals, the present fusion strategy has not fully transformed this complementarity into consistently superior performance. Future work should therefore focus on improving the meta-learning mechanism, such as richer calibration of base-learner probabilities, class-aware fusion, adaptive weighting between spatial and temporal branches, and more robust training strategies, so that the Meta-Learner can deliver a clearer and more reliable performance gain over strong individual base learners.

Furthermore, to validate the overarching efficacy of the proposed system against contemporary architectures, both the base learners and the meta-learner are rigorously benchmarked against established Baseline and adapted State-of-the-Art (SOTA) models within the Network Intrusion Detection Systems (NIDS) domain. Overall, this comprehensive comparative analysis is structured into two primary phases: **an evaluation of single-architecture models against the proposed base learners**, and **a comparative study of the proposed meta-learner against recent SOTA ensemble learning frameworks.**

6.1 Base Learners

6.1.1 Simulation Method

For the single-architecture evaluation, the selected baselines encompass traditional rule-based algorithms and Machine Learning classifiers, including Decision Tree, K-Nearest Neighbors (KNN), Logistic Regression, Support Vector Machines (SVM), and Random Forest, alongside standard Graph Neural Networks (specifically, the Graph Attention Network - GAT). Additionally, the two proposed base learners are benchmarked against graph-based adapted SOTA NIDS architectures, namely Dynamic IDS [17] and E-GraphSAGE [15]. **These two SOTA models were reproduced as adapted baselines using the same preprocessing, data splits, metrics, and repeated-run protocol as the proposed models. Since the target datasets do not follow the graph schema assumed in the original studies, the dataset-interface stage was adapted to construct compatible graphs, including node/edge mapping, edge features, and input dimensions.** The core graph-learning logic, message-passing mechanism, and classification pipeline of each baseline were otherwise preserved as closely as possible to their respective original publications. To ensure statistical reliability and mitigate training variance, each model was independently trained and evaluated three times.

6.1.2 Use Case 1: HTTP-based Attack - IoT-IDS-Zwave-2025

Table 6.1: Performance Evaluation of Intrusion Detection Models - Use Case 1

Model	Macro Precision	Macro Recall	Macro F1	AUC-ROC
GAT Embedding + Raw + XGBoost (Proposed)	**0.9333 ± 0.0018**	0.9162 ± 0.0040	**0.9205 ± 0.0020**	**0.9984 ± 0.0003**
GAT Embedding + XGBoost (Proposed)	0.9287 ± 0.0085	**0.9210 ± 0.0083**	0.9202 ± 0.0085	0.9983 ± 0.0003
Residual CNN BiLSTM (Proposed)	0.8695 ± 0.0160	0.9184 ± 0.0033	0.8794 ± 0.0138	0.9906 ± 0.0053
SVM	0.7635 ± 0.0325	0.7070 ± 0.0312	0.6858 ± 0.0172	0.9802 ± 0.0022
KNN	0.8057 ± 0.0000	0.7578 ± 0.0000	0.7472 ± 0.0000	-
Logistic Regression	0.7429 ± 0.0286	0.7253 ± 0.0137	0.6979 ± 0.0191	0.9855 ± 0.0037
Random Forest	0.8433 ± 0.0027	0.8848 ± 0.0005	0.8539 ± 0.0017	0.9337 ± 0.0000
Decision Tree	0.8483 ± 0.0029	0.8874 ± 0.0042	0.8580 ± 0.0029	0.9269 ± 0.0000
XGBoost	0.8445 ± 0.0000	0.8788 ± 0.0000	0.8501 ± 0.0000	0.9967 ± 0.0000
GAT	0.8797 ± 0.0164	**0.9210 ± 0.0053**	0.8870 ± 0.0197	0.9715 ± 0.0037
Flow-based Dynamic IDS	0.8594 ± 0.0119	0.9130 ± 0.0071	0.8700 ± 0.0076	0.9970 ± 0.0010
E-GraphSAGE	0.7502 ± 0.0572	0.8024 ± 0.0255	0.7391 ± 0.0590	0.9499 ± 0.0256

The experimental results for Use Case 1 demonstrate the quality of the proposed base learners compared to traditional baseline models and current adapted baselines methods. Most notably, the GAT Embedding + Raw Features + XGBoost model achieves the highest performance across all evaluation metrics, yielding a Macro F1 score of 92.05% and an AUC-ROC of 99.84%. The variant excluding raw features (GAT Embedding + XGBoost) follows closely, securing a Macro F1 of 92.02%. This strongly indicates that utilizing Graph Attention Networks (GAT) to extract spatial embeddings provides a high-quality feature space. Consequently, this enables tree-based classification algorithms like XGBoost to effectively identify complex attack patterns that are difficult to represent using raw features alone. Regarding the temporal approach, the Residual CNN-BiLSTM model also exhibits impressive reliability, achieving a Macro F1 of 87.94% and a Recall of approximately 91.84%, which affirms the sliding window technique's capability to robustly capture time-series dependencies.

When compared to the baseline group, the performance gap becomes even more pronounced. Traditional machine learning algorithms, such as SVM, KNN, and Logistic Regression, struggle significantly with the classification task, yielding modest Macro F1 scores ranging from 68.58% to 74.72%. Tree-based models (Decision Tree, Random Forest) and the standalone XGBoost algorithm achieve relatively better results (around 85%); however, a substantial performance gap remains when compared to the proposed deep learning-integrated architectures.

Particularly, when evaluated against the reproduced-and-adapted graph-based baselines under the same experimental protocol, the proposed method demonstrates strong competitiveness. Among the two representative graph-based architectures—

Flow-based Dynamic IDS and E-GraphSAGE—the highest recorded outcome belongs to Flow-based Dynamic IDS, with a Macro F1 of 87.00% and an AUC-ROC of 99.70%. Consequently, the proposed spatial architecture (GAT + XGBoost) improves the F1 score by over 5% compared with this adapted baseline in Use Case 1. Concurrently, it shows greater stability than the adapted E-GraphSAGE architecture, which achieved an F1 of 73.91% with a relatively large standard deviation of 0.0590.

6.1.3 Use Case 2: Application-based Attack - IoT-IDS-Zwave-2025

Table 6.2: Performance Evaluation of Intrusion Detection Models - Use Case 2

Model	Macro Precision	Macro Recall	Macro F1	AUC-ROC
GAT Embedding + Raw + XGBoost (Proposed)	0.9504 ± 0.0058	$\mathbf{0.9682 \pm 0.0033}$	0.9544 ± 0.0057	0.9987 ± 0.0007
GAT Embedding + XGBoost (Proposed)	0.9385 ± 0.0146	0.9502 ± 0.0224	0.9369 ± 0.0229	0.9968 ± 0.0005
Residual CNN BiLSTM (Proposed)	0.9218 ± 0.0151	0.9581 ± 0.0021	0.9333 ± 0.0105	0.9996 ± 0.0001
SVM	0.9264 ± 0.0115	0.8780 ± 0.0050	0.8772 ± 0.0033	0.9987 ± 0.0003
Logistic Regression	0.9188 ± 0.0288	0.9022 ± 0.0315	0.8878 ± 0.0348	0.9978 ± 0.0009
Random Forest	0.8159 ± 0.0000	0.9321 ± 0.0000	0.8251 ± 0.0000	0.9392 ± 0.0000
Decision Tree	0.8126 ± 0.0000	0.8687 ± 0.0000	0.8160 ± 0.0000	0.8805 ± 0.0000
XGBoost	$\mathbf{0.9531 \pm 0.0000}$	0.9668 ± 0.0000	$\mathbf{0.9568 \pm 0.0000}$	$\mathbf{0.9998 \pm 0.0000}$
GAT	0.9497 ± 0.0009	$\mathbf{0.9682 \pm 0.0017}$	0.9548 ± 0.0007	0.9996 ± 0.0000
Flow-based Dynamic IDS	0.9188 ± 0.0163	0.9582 ± 0.0050	0.9256 ± 0.0155	0.9995 ± 0.0004
E-GraphSAGE	0.7441 ± 0.0482	0.8330 ± 0.0287	0.7651 ± 0.0409	0.9778 ± 0.0132

Transitioning to Use Case 2, empirical results indicate a general performance surge across almost all models compared to Use Case 1. The proposed base architectures maintained excellent reliability: the GAT Embedding + Raw Features + XGBoost model achieved a 95.44% F1-Score and 99.87% AUC, while the Residual CNN-BiLSTM closely followed with an F1 of 93.33%. Notably, the proposed methods outperformed the reproduced-and-adapted graph-based SOTA baselines under the same evaluation setting. Flow-based Dynamic IDS and E-GraphSAGE recorded F1 scores of 92.56% and 76.51%, respectively, suggesting that their graph construction and feature extraction mechanisms are less effective on this target data distribution.

However, when objectively evaluated against the Baseline group, the complex feature integration in Use Case 2 did not yield the same breakthrough as in the previous scenario. Standalone XGBoost (95.68%) and independent GAT (95.48%) achieved slightly higher F1 scores than the proposed hybrid system. This indicates that within the confines of Use Case 2, the raw features inherently possess distinct linear or non-linear separability. Consequently, augmenting XGBoost with complex spatial representations (GAT embeddings) inadvertently created minor informational overhead, failing to push past the performance ceiling already reached by the

standard tree-based algorithm. Nevertheless, the proposed model ensures stability and remains firmly within the top-tier performance bracket with acceptable low error margins.

6.1.4 Use Case 3: CIC IIoT 2025

Table 6.3: Performance Evaluation of Intrusion Detection Models - Use Case 3

Model	Macro Precision	Macro Recall	Macro F1	AUC-ROC
GAT Embedding + Raw + XGBoost (Proposed)	0.7439 ± 0.0080	0.6531 ± 0.0042	0.6809 ± 0.0047	0.9054 ± 0.0037
GAT Embedding + XGBoost (Proposed)	0.7439 ± 0.0076	0.6540 ± 0.0060	0.6835 ± 0.0065	0.9016 ± 0.0062
Residual CNN BiLSTM (Proposed)	$\mathbf{0.7593 \pm 0.0081}$	$\mathbf{0.7459 \pm 0.0321}$	$\mathbf{0.7182 \pm 0.0116}$	$\mathbf{0.9558 \pm 0.0125}$
SVM	0.6340 ± 0.0201	0.5741 ± 0.0120	0.5786 ± 0.0211	0.9164 ± 0.0030
KNN	0.4974 ± 0.0000	0.4771 ± 0.0000	0.4317 ± 0.0000	0.7325 ± 0.0000
Logistic Regression	0.6327 ± 0.0583	0.5743 ± 0.0294	0.5783 ± 0.0391	0.9204 ± 0.0017
Random Forest	0.6477 ± 0.0004	0.6190 ± 0.0001	0.6217 ± 0.0001	0.7975 ± 0.0000
Decision Tree	0.6426 ± 0.0011	0.5787 ± 0.0010	0.5987 ± 0.0012	0.7739 ± 0.0000
XGBoost	0.6678 ± 0.0059	0.5993 ± 0.0018	0.6116 ± 0.0043	0.9241 ± 0.0000
GAT	0.7463 ± 0.0029	0.6382 ± 0.0057	0.6661 ± 0.0060	0.9546 ± 0.0052

The results indicate that Use Case 3 presents a significantly more challenging classification task, evidenced by a universal performance decline across all evaluated models. In this scenario, the Residual CNN-BiLSTM emerged as the top performer, achieving a Macro F1 of 71.82% and an AUC-ROC of 95.58%. This demonstrates that against sophisticated, long-duration stealth attacks, the sequential temporal features extracted via the sliding window and BiLSTM mechanisms are far more resilient and decisive than spatial representations (GAT Embedding + XGBoost).

Nevertheless, the proposed graph-based architecture (GAT Embedding + XGBoost; Macro F1: 68.35%) maintained its superiority by substantially outperforming traditional baselines. Standalone XGBoost, Random Forest, and SVM stagnated with F1 scores ranging from 57.86% to 62.17%, while KNN experienced a near-total loss of classification capability, plummeting to an F1 of 43.17%.

6.2 Meta Learner

6.2.1 Simulation Method

The two best-performing base learners from the three independent experimental runs per use case were selected as inputs for the Meta-Learner. Following the stacking protocol, these base learners were trained on the training set and then used to generate validation-set probability vectors. The Meta-Learner was trained on the concatenated validation probabilities from the spatial and temporal branches. For final testing, the trained base learners generated probability vectors on the independent test set, and these test probabilities were passed to the trained Meta-Learner

to compute the reported ensemble metrics. To validate its efficacy, the proposed ensemble system was rigorously benchmarked against two current State-of-the-Art ensemble architectures: xIDS [19] and a combined framework of XGBoost, LightGBM, CatBoost, and AdaBoost [20].

6.2.2 Results across Use Cases

Table 6.4: Performance comparison of Ensemble Learning models - Use Case 1

Model	Macro Precision	Macro Recall	Macro F1	AUC-ROC
Proposed Meta-Learner (LR)	0.9630	0.9654	0.9638	0.9995
Proposed Meta-Learner (XGBoost)	**0.9705**	**0.9702**	**0.9697**	**0.9997**
xIDS	0.8883	0.9127	0.8967	0.9984
XGBoost, LightGBM, CatBoost, AdaBoost	0.8985	0.9130	0.9028	0.9986

Note: **Proposed Meta-Learner** combines spatial features (GAT Embedding + Raw + XGBoost) and temporal features (Residual CNN-BiLSTM). **LR**: Logistic Regression meta-classifier; **XGBoost**: XGBoost meta-classifier.

For Use Case 1, while state-of-the-art architectures, which predominantly rely on combining gradient boosting decision trees, only achieve a Macro F1 ranging from 89.67% to 90.28%, both Meta-Learner models attain a Macro F1 of approximately 96.38% (using Logistic Regression as the Meta-Learner) and peak at 96.97% (using XGBoost). This substantiates that combining spatial features (from GAT) and temporal features (from CNN-BiLSTM) yields a significantly richer and more valuable set of complementary information compared to merely stacking tree-based algorithms.

Table 6.5: Performance comparison of Ensemble Learning models - Use Case 2

Model	Macro Precision	Macro Recall	Macro F1	AUC-ROC
Proposed Meta-Learner (LR)	0.9456	0.9700	0.9527	0.9848
Proposed Meta-Learner (XGBoost)	0.9554	0.9665	0.9571	0.9981
xIDS	**0.9735**	**0.9747**	**0.9735**	0.9982
XGBoost, LightGBM, CatBoost, AdaBoost	0.9698	0.9573	0.9619	**0.9998**

Note: **Proposed Meta-Learner** combines spatial features (GAT Embedding + Raw + XGBoost) and temporal features (Residual CNN-BiLSTM). **LR**: Logistic Regression meta-classifier; **XGBoost**: XGBoost meta-classifier.

Transitioning to Use Case 2, the data characteristics appear advantageous for pure separation via gradient boosting algorithms. Consequently, the two SOTA models achieve highly impressive F1 scores ranging from 96.19% to 97.35%. Although the proposed system experiences a marginal decline (F1 ranging from 95.27% to 95.71%), these results remain competitive. This approximate 1% variance indicates

a minor trade-off, where the Meta-Learner sacrifices a fraction of its adherence to standard data distributions to preserve the diversity of spatial and temporal representations.

Table 6.6: Performance comparison of Ensemble Learning models - Use Case 3

Model	Macro Precision	Macro Recall	Macro F1	AUC-ROC
Proposed Meta-Learner (LR)	0.8040	**0.7951**	**0.7505**	0.9449
Proposed Meta-Learner (XGBoost)	**0.8310**	0.7405	0.7490	**0.9844**
xIDS	0.7542	0.6640	0.6760	0.9022
XGBoost, LightGBM, CatBoost, AdaBoost	0.8154	0.6690	0.6992	0.9464

Note: **Proposed Meta-Learner** combines spatial features (GAT Embedding + Raw + XGBoost) and temporal features (Residual CNN-BiLSTM). **LR**: Logistic Regression meta-classifier; **XGBoost**: XGBoost meta-classifier.

Crucially, in Use Case 3, when confronted with sophisticated, long-duration stealth attacks, SOTA methods expose significant instability, evidenced by a severe drop in F1 scores to merely 67.60% (xIDS) and 69.92% (Tree Ensemble). Conversely, the two proposed Meta-Learner models demonstrate robustness, maintaining Macro F1 scores between 74.90% and 75.05%. The time-series-focused mechanism (derived from the BiLSTM base learner) effectively compensates for detection gaps, enabling the Meta-Learner to render substantially more stable decisions. In conclusion, the proposed Meta-Learner architecture not only establishes a new SOTA accuracy benchmark in Use Case 1 but also delivers robustness and generalization across adverse network environments.

Results Discussion

The experimental results above show that the proposed ensemble architecture provides robust intrusion classification across IoT and IIoT traffic scenarios. In Use Case 1, the XGBoost meta-learner achieves the strongest overall performance, showing that HTTP-based DDoS attacks benefit from the joint use of spatial graph-based predictions and temporal sequence-based predictions. In Use Case 2, the proposed model remains competitive, although tree-based ensemble baselines obtain slightly better macro-level scores, suggesting that some application-based DDoS patterns are already well captured by conventional flow-level features. In Use Case 3, the proposed meta-learners achieve the best Macro F1 scores, indicating that spatial–temporal fusion is especially useful when industrial IoT attacks are harder to separate and class imbalance affects model stability.

The main strength of the system is its complementary design. The GAT–XGBoost branch models structural relationships among temporally related flows, while the

Residual CNN–BiLSTM branch captures chronological attack behavior through sliding-window sequence modeling. By fusing their predictive distributions, the meta-learner reduces dependence on a single representation type and improves robustness across different traffic conditions. The results also show that the choice of meta-learner can be adapted to the objective: XGBoost is effective for nonlinear fusion, whereas logistic regression can provide stable recall-oriented behavior in more difficult settings.

7.1 Overall

While offline evaluation metrics such as F1-score, Precision, and Recall validate the theoretical predictive accuracy of the proposed architecture, assessing its true viability in a production-like cybersecurity environment necessitates the processing of continuous, real-time network traffic streams.

To achieve this, the entire data pipeline and inference architecture have been deployed using containerization technology, specifically via Docker and Docker Compose. Containerization is deliberately chosen to ensure environmental consistency, component isolation, and reproducibility across different infrastructure setups. By modularizing each core component into distinct containers, the deployment mimics a modern, microservices-based enterprise architecture.

The deployed ecosystem is designed to handle an end-to-end streaming data lifecycle. It begins with the ingestion of simulated real-time network flows into a message broker, followed by high-throughput stream preprocessing. Subsequently, the containerized ensemble system—comprising the spatial-temporal feature extractors and the Meta-Learner—executes real-time threat classification on the processed micro-batches. Finally, the prediction results are aggregated into a time-series database and visualized through an interactive monitoring dashboard. Ultimately, this Docker-based deployment strategy bridges the gap between theoretical machine learning research and practical cybersecurity operations, rigorously demonstrating the system's capability to maintain low-latency inference and high throughput in a dynamic network environment.

7.2 Deployment Diagram

Figure 7.1 illustrates the high-level deployment architecture of the proposed ensemble Network Intrusion Detection System. To ensure scalability and environmental consistency, the system is designed as a decoupled, event-driven microservices architecture orchestrated via Docker Compose within an isolated bridge network (`nids_net`).

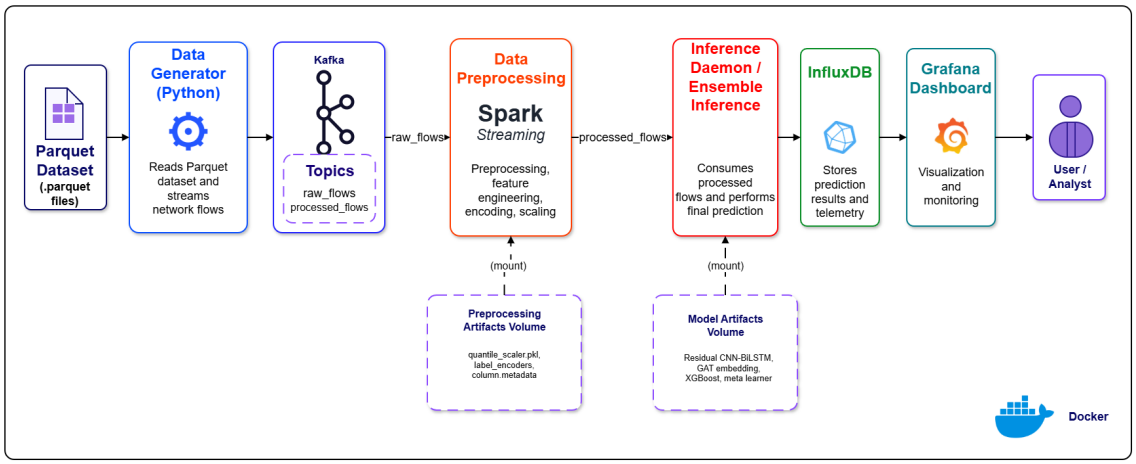

Figure 7.1: Deployment diagram of the proposed ensemble architecture.

Initially, a host-based Data Generator simulates real-time network traffic by sequentially pushing raw flow records to an Apache Kafka broker. A Spark Structured Streaming container acts as the preprocessing engine, continuously consuming these raw flows, applying distributed transformations (e.g., one-hot encoding and quantile scaling), and publishing the refined data back to a secondary Kafka topic.

Subsequently, the Inference Daemon—the core computational container—consumes these processed packets to construct dynamic temporal windows and spatial flow-based graphs. This daemon executes proposed ensemble architectures to detect anomalous behaviors. Finally, the inference results, along with their metadata, are batch-written into InfluxDB, a high-performance time-series database, enabling Grafana to query and render real-time security alerts and traffic metrics on an interactive dashboard. The details for each container in deployment phase are illustrated bellow.

7.2.1 Data Generator Simulation

To facilitate controlled, reproducible, and standardized evaluation within this deployment pipeline, a custom Data Generator component (`data_generator.py`) is implemented. This module serves as the primary data ingestion point, bridging the gap between static historical datasets and dynamic, real-time data streams.

The generator operates by reading preprocessed network flows stored locally in Parquet format. To ensure strict temporal accuracy—a critical requirement for the downstream spatial-temporal ensemble model—the script normalizes the `timestamp_start` attribute from ISO 8601 strings into high-precision Unix timestamps (floating-point seconds). The entire dataset is subsequently sorted in chronological order to prevent out-of-sequence packet transmission and maintain the integrity of time-series patterns.

A notable feature of this simulation module is its ability to mathematically replicate the realistic inter-arrival times of network flows. For each sequentially processed flow f_i, the script calculates the actual temporal delta (Δt_{real}) relative to the preceding flow. To accommodate both realistic testing and accelerated evaluation scenarios, a configurable speed factor parameter (S_f) is introduced. The system computes a simulated processing delay ($\Delta t_{\text{sim}} = \frac{\Delta t_{\text{real}}}{S_f}$) and intentionally suspends thread execution for this exact duration before transmitting the next record to the Kafka topic $\mathcal{K}$.

This mechanism enables the deployment ecosystem to process massive traffic volumes at an accelerated rate (e.g., 10,000x normal speed) without distorting the chronological sequencing essential for sequence learning and graph construction. Finally, the generator serializes each flow record into a structured JSON object and acts as a producer, continuously publishing the stream to the `raw_flows` topic on the Apache Kafka broker.

Algorithm 7.1: Real-time Traffic Simulation with Configurable Speed Factor

Input: $\mathcal{D} = \{f_i\}_{i=1}^{N}$: preprocessed dataset of network flows, where $f_i = (\mathbf{x}_i, \mathbf{m}_i, y_i)$ and metadata $\mathbf{m}_i$ contains timestamp t_i; $S_f \in \mathbb{R}^+$: speed factor; $\mathcal{K}$: target Apache Kafka topic.

Output: Continuous data stream to $\mathcal{K}$ preserving relative temporal patterns.

$\mathcal{D} \leftarrow \text{Sort}(\mathcal{D})$ chronologically such that $t_1 \leq t_2 \leq \ldots \leq t_N$;

$t_{\text{prev}} \leftarrow t_1$; `// Initialize reference timestamp`

for $i \leftarrow 1$ **to** N **do**

- $t_{\text{curr}} \leftarrow t_i$;
- $\Delta t_{\text{real}} \leftarrow t_{\text{curr}} - t_{\text{prev}}$;
- $\Delta t_{\text{sim}} \leftarrow \frac{\Delta t_{\text{real}}}{S_f}$;
- **if** $\Delta t_{\text{sim}} > 0$ **then**
  - $\text{Wait}(\Delta t_{\text{sim}})$ seconds;
- $\text{payload} \leftarrow \text{SerializeToJSON}(f_i)$;
- $\text{PublishToKafka}(\mathcal{K}, \text{payload})$;
- $t_{\text{prev}} \leftarrow t_{\text{curr}}$; `// Update reference time`

7.2.2 Kafka

In the real-time data streaming architecture of the proposed NIDS, Apache Kafka serves as the central distributed message broker, responsible for asynchronous event orchestration, backpressure management, and ensuring data immutability as network flows transit between decoupled microservices. The messaging infrastructure is

deployed as an isolated microservice cluster within the internal network, comprising a Zookeeper node to synchronize cluster metadata and a single Kafka broker optimized with a replication factor of one. To securely separate internal data processing from external data ingestion, the broker implements a dual-listener configuration. The internal listener facilitates high-speed, isolated communication for Dockerized analytical engines, whereas the external listener allows the host-based Data Generator to continuously ingest raw traffic into the pipeline.

The broker orchestrates two distinct sequential topics. The primary ingestion topic, designated as `raw_flows`, directly receives serialized JSON payloads from the Data Generator and acts as the raw traffic buffer for the Spark Structured Streaming engine. Subsequently, the `processed_flows` topic serves as the output of the preprocessing tier, receiving the normalized feature vectors from Spark and functioning as a standardized data feed for the downstream Inference Daemon.

To guarantee high availability and prevent data loss during transient network interruptions or container restarts, strict consumer configurations are enforced. By setting the auto-offset reset policy to the earliest available record, the preprocessing and inference modules systematically consume all unread messages from the beginning of the partition upon connection recovery. Furthermore, the system incorporates an automated, continuous retry strategy, empowering the dependent daemons to gracefully survive temporary broker unavailability without catastrophic failures, thereby maintaining the resilience of the entire intrusion detection pipeline.

7.2.3 Spark Streaming

The data transformation stage is orchestrated by an Apache Spark Structured Streaming container, functioning as the high-throughput preprocessing engine of the proposed NIDS pipeline. Deployed within the isolated Docker network, this component continuously ingests raw, serialized network flows from the `raw_flows` Kafka topic. To achieve low-latency processing while executing complex data manipulations, the architecture leverages Pandas User-Defined Functions (UDFs) via Apache Arrow, which enables highly optimized, vectorized operations across data micro-batches. This approach effectively bridges the distributed processing capabilities of Apache Spark with the extensive data science ecosystem of Python, ensuring that real-time streams are processed with the same rigor as historical training data.

Within each computational micro-batch, the engine systematically applies a sequence of trained machine learning transformations essential for downstream inference. The worker node initially filters out redundant attributes and dynamically

parses string-encoded arrays to apply one-hot encoding on categorical features, such as log data types and source/destination network protocols, utilizing pre-trained `MultiLabelBinarizer` files. Concurrently, continuous numerical network features are standardized using a pre-fitted `QuantileTransformer` to mitigate the impact of outliers and properly scale the feature space. Following these complex transformations, the system strictly enforces a predefined output schema comprising 138 explicitly typed features, securely mapping and casting data types to prevent runtime anomalies in the subsequent prediction phase. Finally, the standardized feature vectors are reserialized into JSON payloads and continuously published to the `processed_flows` topic, with robust fault tolerance and exactly-once processing semantics maintained through stateful offset checkpointing.

7.2.4 Ensemble Inference

The core computational intelligence of the proposed NIDS pipeline is encapsulated within the Inference Daemon, a containerized microservice responsible for executing the hybrid spatial-temporal ensemble model in real time. To ensure high-performance execution and seamless hardware acceleration, the service is built upon the official PyTorch runtime environment with CUDA 12.1 support (`pytorch/pytorch:2.5.1-cuda12.1-cudnn9-runtime`). Rather than embedding massive model weights directly into the Docker image, the system utilizes read-only volume mounts to dynamically load pre-trained artifacts—including PyTorch state dictionaries and XGBoost model configurations—from the host filesystem. This design pattern strictly adheres to the principle of stateless containerization, allowing the inference engine to remain lightweight while facilitating effortless model updates without requiring image rebuilds.

Upon initializing the models and connecting to the messaging broker, the daemon continuously consumes standardized feature vectors from the `processed_flows` topic. To bridge the gap between individual network packets and the complex input requirements of deep learning models, a stateful `BufferManager` is implemented. This module aggregates incoming messages into configurable micro-batches (`CHUNK_SIZE=256`) and subsequently transforms them into dual mathematical representations. It constructs temporal sliding windows with a configured length of 10 time steps for the sequence learning branch, while simultaneously building dynamic spatial graphs comprising up to 50 recent nodes, interlinked based on Jaccard similarities of IP addresses and ports within a 30-second temporal threshold.

Once the data structures are aligned, the `EnsembleManager` orchestrates

the forward propagation through the hybrid pipeline. To maximize throughput and minimize Video RAM (VRAM) consumption during real-time deployment, the entire inference process is executed within a gradient-disabled context (`torch.no_grad()`), and all neural network modules are strictly locked in evaluation mode (`eval()`) to neutralize stochastic behaviors such as Dropout. The temporal features are processed by the Residual CNN-BiLSTM-Attention branch, while the topological graphs are concurrently fed into the GAT-XGBoost branch. The probabilistic outputs from both base learners are then concatenated and fed into the XGBoost Meta-Learner, which synthesizes the final threat classification. Ultimately, the daemon systematically packages these predictions alongside essential metadata and synchronously batch-writes the results into the InfluxDB time-series database, laying the foundation for downstream operational monitoring.

7.2.5 InfluxDB

To persist and manage the continuous stream of classification results generated by the hybrid ensemble model, InfluxDB is integrated into the deployment architecture as the primary time-series database. Specifically, the system utilizes the `influxdb:2.7` container image, configured within the isolated bridge network to ensure secure and low-latency data ingestion directly from the Inference Daemon. Recognizing the ephemeral nature of containerized environments, the database is equipped with persistent Docker volume mapping (`influxdb_data`) to guarantee long-term data retention across system reboots or technical failures. Within the operational pipeline, the Inference Daemon leverages a synchronous write API to batch-insert predictions, an approach that significantly optimizes the database's write throughput and minimizes network I/O overhead during high-speed traffic simulation.

The data schema within InfluxDB is meticulously engineered to accommodate the high-cardinality and high-velocity nature of network traffic logging. Each classification event is encapsulated as a discrete data point under the `network_flow` measurement. To facilitate rapid querying and multidimensional aggregation in the downstream visualization layer, critical identifiers such as the unique `flow_id`, `src_ip`, and `dst_ip` are explicitly designated as indexed tags. Conversely, the core analytical metrics, including the algorithm's `predicted_label` and Kafka offset tracking variables, are stored as unindexed fields to conserve indexing memory. Crucially, to preserve the exact chronological sequence of the accelerated network simulation, each record is timestamped with nanosecond precision (`WritePrecision.NS`) derived directly from the original flow metadata. This rigorous time-series architecture ensures that the system can effortlessly support

complex temporal queries and real-time operational monitoring of malicious network behaviors.

7.2.6 Grafana

To provide comprehensive, real-time observability over the network security environment, Grafana is deployed as the visualization and monitoring layer of the proposed architecture. Operating as a containerized microservice based on the official `grafana/grafana:latest` image, it is exposed on port `3000` and integrated directly into the isolated `nids_net` bridge network. This architectural placement ensures secure, high-bandwidth communication with the underlying InfluxDB data store while providing a web-based interface for security analysts.

Within the deployment orchestration, Grafana exhibits a strict startup dependency on the InfluxDB container (`depends_on: influxdb: condition: service_healthy`), guaranteeing that the visualization engine initializes only after the time-series database is fully operational and capable of accepting queries. To query the real-time inference results, Grafana utilizes the Flux query language to aggregate, filter, and process the high-velocity data points stored in the `nids_bucket`.

Furthermore, to maintain operational continuity, the deployment employs persistent volume mapping (`grafana_data`). This stateless-container paradigm ensures that all custom dashboard layouts, data source configurations, data mappings, and administrative credentials (`GF_SECURITY_ADMIN_USER`) are safely preserved across container restarts or system updates. Ultimately, this component translates raw, multidimensional classification data into an intuitive, interactive graphical interface, empowering administrators to continuously monitor the network's security posture and swiftly identify emerging threat patterns.

7.3 Dashboard Visualization

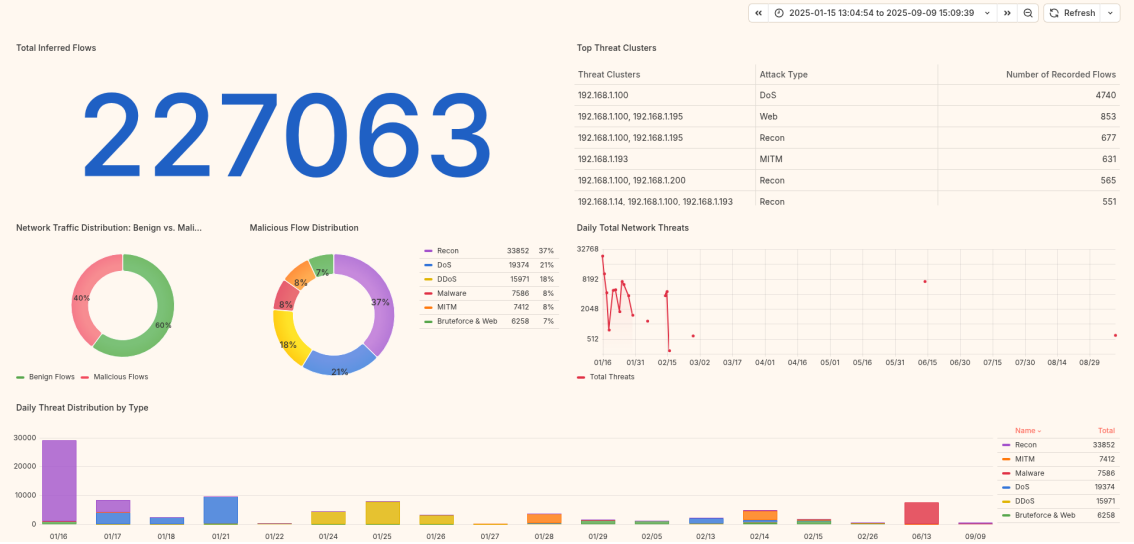

Figure 7.2: Dashboard Visualization by Grafana

Figure 7.2 presents the comprehensive monitoring dashboard developed to visualize the output of the proposed intrusion detection system. Serving as the primary administrative interface, the dashboard provides an overview of the network's security posture by transforming complex classification outputs into intuitive visual metrics. It systematically aggregates critical security data, including overall network traffic volumes, the proportional distribution of benign versus malicious flows, specific threat typologies, and the temporal evolution of attack patterns. In addition, the dashboard supports querying results over specific time ranges, allowing analysts to focus on particular operational windows or incident periods. Through interactive legend controls, users can also isolate, inspect, and compare individual attack types in greater detail. By implementing this dashboard, the system enables rapid situational awareness and effective threat assessment. A detailed functional analysis of each individual visualization component within the dashboard is provided in the subsequent subsections.

7.3.1 Chart: Total Inferred Flows

Figure 7.3: Total inferred flows displayed in the Grafana dashboard.

Figure 7.3 presents the total number of network flows that have been processed and classified by the deployed inference pipeline. This chart functions as a high-level throughput indicator, allowing administrators to quickly verify that the Data Generator, Kafka stream, Spark preprocessing stage, Inference Daemon, and InfluxDB storage layer are operating continuously. A sudden stagnation in this value may indicate an interruption in data ingestion, preprocessing, or database writing, while a rapid increase confirms that the simulation is actively producing high-volume traffic. Therefore, this panel serves as the first operational health check for the end-to-end streaming architecture before analysts proceed to more detailed threat-oriented visualizations.

7.3.2 Chart: Top Threat Clusters

Top Threat Clusters

Threat Clusters	Attack Type	Number of Recorded Flows
192.168.1.100	DoS	4740
192.168.1.100, 192.168.1.195	Web	853
192.168.1.100, 192.168.1.195	Recon	677
192.168.1.193	MITM	631
192.168.1.100, 192.168.1.200	Recon	565
192.168.1.14, 192.168.1.100, 192.168.1.193	Recon	551
192.168.1.18, 192.168.1.100, 192.168.1.193	Recon	548
192.168.1.195, 142.250.69.142	Web	542
192.168.1.53, 192.168.1.100	Recon	538
192.168.1.10, 192.168.1.100, 192.168.1.193	Recon	525
192.168.1.13, 192.168.1.100, 192.168.1.193	Recon	514
192.168.1.1	MITM	509
192.168.1.100, 192.168.1.200	DoS	488
192.168.1.15, 192.168.1.100, 192.168.1.193	Recon	479
192.168.1.100, 192.168.1.57	DoS	478

Figure 7.4: Top threat clusters detected by the Grafana dashboard.

Figure 7.4 summarizes the most frequent malicious threat clusters identified by the deployed intrusion detection pipeline. This chart helps analysts prioritize the dominant attack categories in the monitored traffic, enabling faster triage and more focused investigation of recurring threat patterns. By ranking threats according to their observed frequency, the dashboard highlights which attack groups require

immediate attention from security operators. It also supports comparative analysis across monitoring sessions, since changes in the ordering of clusters can reveal shifts in attacker behavior or the emergence of new dominant threat families.

7.3.3 Chart: Benign vs. Malicious

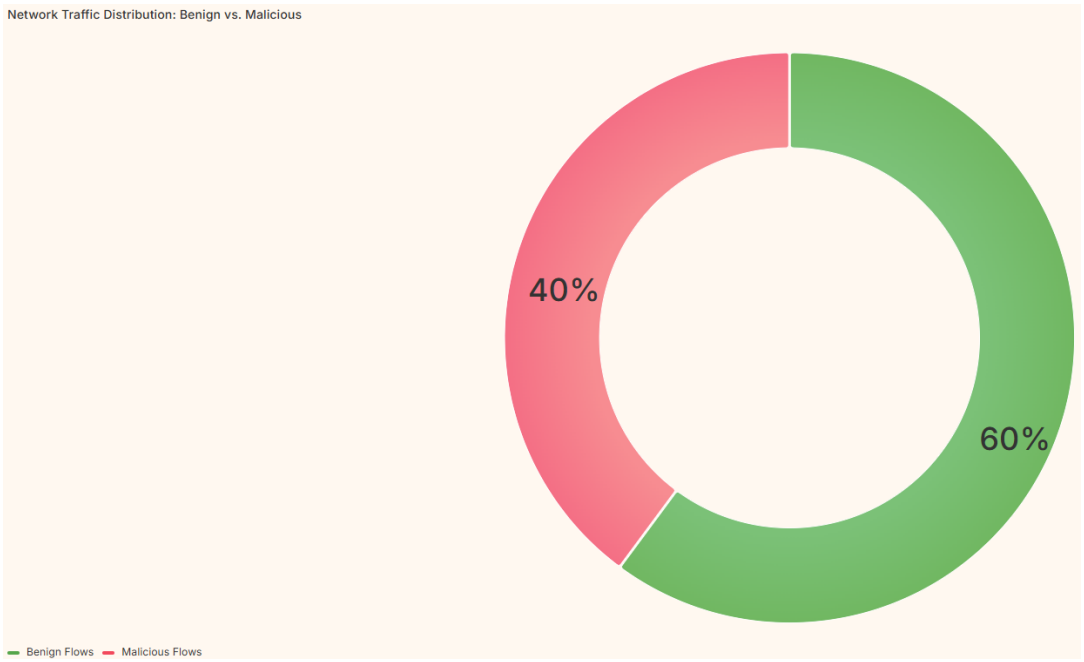

Figure 7.5: Benign and malicious flow proportions in the Grafana dashboard.

Figure 7.5 visualizes the proportional relationship between benign and malicious flows detected by the deployed NIDS. This chart provides an immediate summary of the current security posture, helping administrators quickly determine whether monitored traffic is dominated by normal behavior or active attack activity. When the malicious portion grows substantially, the system can be interpreted as operating under a more hostile traffic condition, requiring closer inspection of the corresponding attack categories. Conversely, a mostly benign distribution helps validate that the dashboard is not only displaying alerts but also preserving the broader context of normal network operation.

7.3.4 Chart: Malicious Flow Distribution

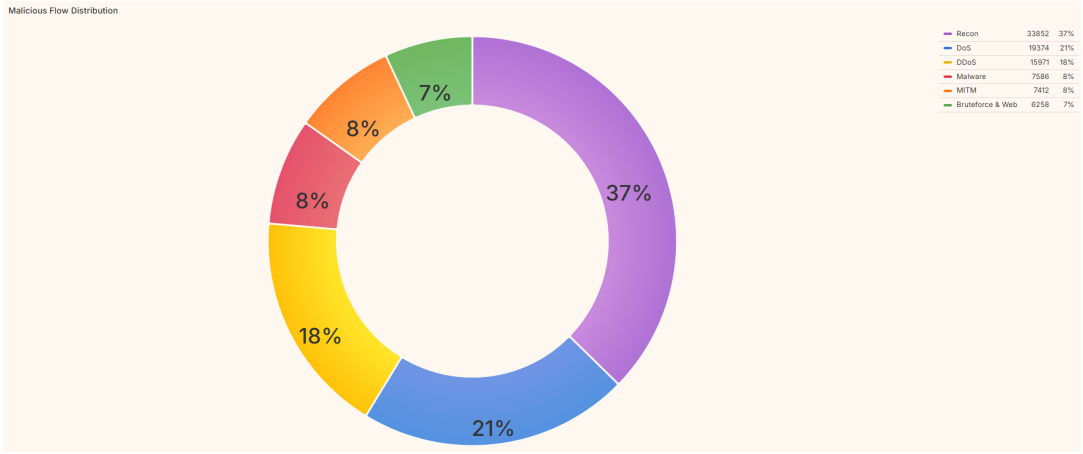

Figure 7.6: Distribution of malicious flows by attack category in the Grafana dashboard.

Figure 7.6 illustrates how malicious flows are distributed across different detected attack categories. By decomposing the overall malicious traffic into specific threat types, this chart supports more detailed incident analysis and helps administrators identify which attack behaviors contribute most significantly to the observed risk level. The distribution also makes it easier to distinguish between broad, multi-class attack activity and incidents dominated by a single threat type. Such information is valuable for selecting appropriate response actions, because different attack families may require different mitigation strategies, firewall rules, or investigation priorities.

7.3.5 Chart: Daily Total Network Threats

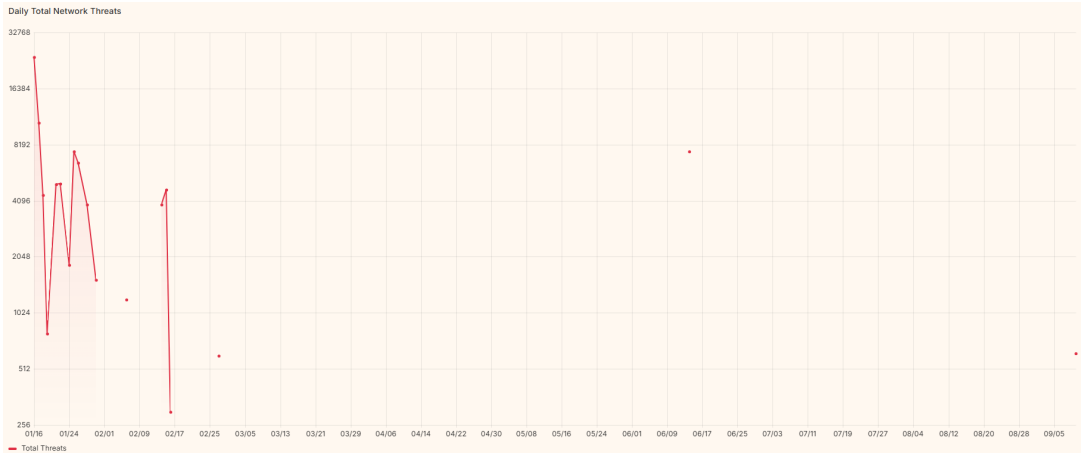

Figure 7.7: Daily total network threats visualized in the Grafana dashboard.

Figure 7.7 presents the daily volume of detected network threats over the monitored time horizon. This time-series view helps administrators observe fluctuations in malicious activity, identify abnormal spikes, and assess whether the deployed

NIDS is capturing evolving attack intensity across different days. Peaks in this chart may correspond to bursty attack campaigns, stress-test intervals, or specific simulation periods in which malicious samples are concentrated. As a result, the visualization supports temporal correlation with operational events, model logs, or infrastructure metrics to better understand when the system experiences the highest threat pressure.

7.3.6 Chart: Daily Threat Distribution by Type

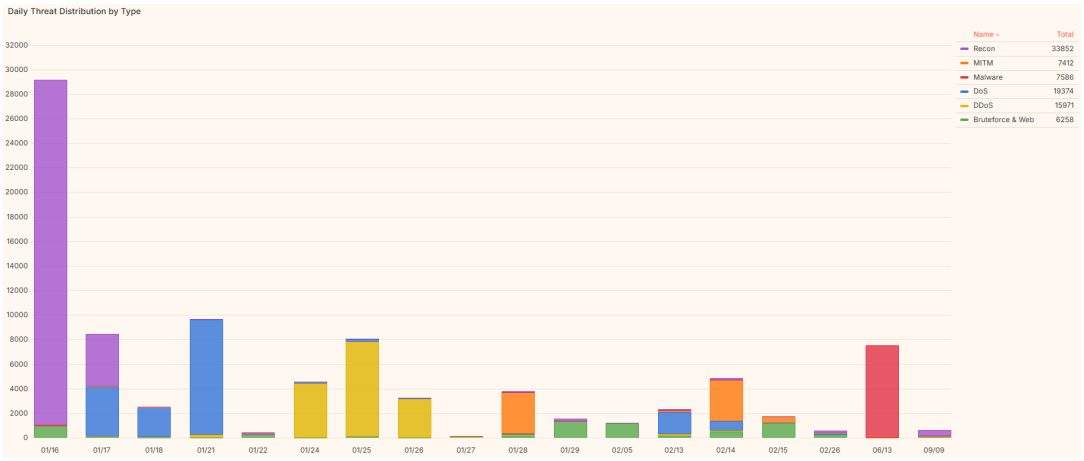

Figure 7.8: Daily threat distribution by attack type in the Grafana dashboard.

Figure 7.8 breaks down daily malicious traffic according to the detected attack categories. This visualization enables analysts to compare how different threat types evolve over time, supporting trend analysis and helping identify days where specific attacks become unusually dominant. Unlike the total daily threat count, this chart preserves the composition of the attacks, showing whether increases in malicious activity are caused by one category or by multiple concurrent threat classes. The stacked temporal view is especially useful for post-incident review, because analysts can isolate attack types through the legend and inspect how each category contributes to the overall threat landscape during a selected time window.

7.4 Deployment Conclusion

The deployment phase has demonstrated the practical deployment of the proposed ensemble-based NIDS within a production-like, containerized streaming environment. By integrating Docker Compose, Apache Kafka, Spark Streaming, the Inference Daemon, InfluxDB, and Grafana, the system transforms the offline learning architecture into an operational pipeline capable of ingesting, preprocessing, classifying, storing, and visualizing network flows in near real time. The deployment design validates that the proposed model is not only suitable for experimental evaluation, but can also be embedded into a modular cybersecurity monitoring

workflow.

The containerized architecture provides several important practical benefits. Kafka decouples data ingestion from downstream processing and improves resilience against temporary service interruptions, while Spark Streaming standardizes raw traffic into model-ready feature vectors using the same transformation logic applied during training. The Inference Daemon then executes the hybrid temporal and spatial ensemble model under an optimized runtime configuration, and InfluxDB stores the prediction outputs as time-series records that preserve the chronological structure of the simulated traffic.

Finally, the Grafana dashboard illustrates the operational value of the deployment by converting prediction results into interpretable monitoring panels. Through aggregate counters, threat-cluster rankings, benign-versus-malicious summaries, attack-type distributions, and temporal trend charts, administrators can observe both the overall security posture and detailed attack dynamics. Overall, this deployment phase solves the gap between model development and real-world applicability, showing that the proposed NIDS can support continuous monitoring, rapid situational awareness, and practical threat analysis in a scalable microservices-based environment.

CHAPTER 8. CONCLUSIONS

8.1 Limitations and Future Work

The proposed architecture still has several limitations. The graph branch introduces additional computational cost because it requires flow-based graph construction, which may be challenging for large-scale or real-time deployment. The temporal branch depends on window size and flow ordering, which can affect performance under sparse, bursty, or partially reordered traffic. Moreover, the evaluation is limited to three use cases from two datasets, so additional validation on broader network environments is needed to assess generalization. Future work will therefore focus on reducing computational complexity, improving inference latency for large-scale network flows, and extending the architecture toward proactive threat forecasting and attack provenance analysis. Promising directions include incremental graph construction, adaptive temporal windowing, drift-aware learning [43], model compression, and interpretable decision fusion for practical NIDS deployment.

8.2 Conclusion

In conclusion, the thesis presented an end-to-end ensemble learning architecture for intrusion attack classification. The proposed system combines a GAT embedding XGBoost learner for spatial traffic relationships and a residual CNN–BiLSTM learner for temporal attack patterns. A stacked meta-learner fuses both predictive signals and improves robustness across different IoT and IIoT use cases.

Experiments on IoT-IDS-Zwave-2025 and CIC IIoT 2025 show that the proposed method is competitive with traditional machine-learning baselines, graph-based IDS models, and recent ensemble IDS frameworks. The results confirm that combining spatial and temporal representations is a promising direction for building more reliable NIDS models in heterogeneous traffic environments.

REFERENCE

[1] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, *Graph attention networks*, 2018. arXiv: 1710.10903 [stat.ML]. Accessed: Jun. 3, 2026. [Online]. Available: https://arxiv.org/abs/1710.10903

[2] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794. DOI: 10.1145/2939672.2939785

[3] M. Shafi and A. H. Lashkari, "Toward generating a large-scale iot-zwave intrusion detection dataset: Smart device profiling, intruders behavior, and traffic characterization," *Internet of Things*, vol. 34, p. 101 747, 2025. DOI: 10.1016/j.iot.2025.101747

[4] A. Firouzi, S. Dadkhah, S. A. Maret, and A. A. Ghorbani, "Datasense: A real-time sensor-based benchmark dataset for attack analysis in iiot with multi-objective feature selection," *Electronics*, vol. 14, p. 4095, 2025.

[5] H.-J. Liao, C.-H. R. Lin, Y.-C. Lin, and K.-Y. Tung, "Intrusion detection system: A comprehensive review," *Journal of Network and Computer Applications*, vol. 36, no. 1, pp. 16–24, 2013. DOI: 10.1016/j.jnca.2012.09.004

[6] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016. DOI: 10.1109/COMST.2015.2494502

[7] L. Le, *Vietnam cybersecurity report 2025*, Nhan Dan newspaper, Nhan Dan newspaper, Jan. 2026. Accessed: Jun. 3, 2026. [Online]. Available: https://nhandan.vn/viet-nam-phai-doi-mat-voi-khoang-552000-cuoc-tan-cong-mang-trong-nam-2025-post936719.html

[8] H. Hicklen, *Cyber attacks are on the rise: How businesses are adapting*, Clutch, Clutch, May 2026. Accessed: Jun. 28, 2026. [Online]. Available: https://clutch.co/resources/cybersecurity-trends-2025

[9] Z. Xu et al., *Deep learning-based intrusion detection systems: A survey*, 2025. arXiv: 2504.07839 [cs.CR]. Accessed: Jun. 3, 2026. [Online]. Available: https://arxiv.org/abs/2504.07839

[10] R. Zhao, X. Deng, Z. Yan, J. Ma, Z. Xue, and Y. Wang, "Mt-flowformer: A semi-supervised flow transformer for encrypted traffic classification," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, Aug. 2022, pp. 2576–2584. DOI: 10.1145/3534678.3539314

[11] A. Naeem, J. Ahmad, M. A. Khan, A. A. Khattak, and M. S. Khan, *Efficient iot intrusion detection with an improved attention-based cnn-bilstm architecture*, 2026. arXiv: 2503.19339 [cs.CR]. Accessed: Jun. 2, 2026. [Online]. Available: https://arxiv.org/abs/2503.19339

[12] B. Ahmad, Z. Wu, Y. Huang, and S. U. Rehman, "Enhancing the security in iot and iiot networks: An intrusion detection scheme leveraging deep transfer learning," *Knowledge-Based Systems*, vol. 305, p. 112 614, 2024. DOI: 10.1016/j.knosys.2024.112614

[13] F. Nie, W. Liu, G. Liu, and B. Gao, "M2VT-IDS: A multi-task multi-view learning architecture for designing iot intrusion detection system," *Internet of Things*, vol. 25, p. 101 102, 2024. DOI: 10.1016/j.iot.2024.101102

[14] S. Hizal, U. Cavusoglu, and D. Akgun, "A novel deep learning-based intrusion detection system for iot ddos security," *Internet of Things*, vol. 28, p. 101 336, 2024. DOI: 10.1016/j.iot.2024.101336

[15] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, "E-graphsage: A graph neural network based intrusion detection system for iot," in *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*, Budapest, Hungary: IEEE, Apr. 2022, pp. 1–9. DOI: 10.1109/noms54207.2022.9789878 [Online]. Available: http://dx.doi.org/10.1109/NOMS54207.2022.9789878

[16] K. Hsieh et al., "NetVigil: Robust and Low-Cost anomaly detection for East-West data center security," in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, Santa Clara, CA: USENIX Association, Apr. 2024, pp. 1771–1789, ISBN: 978-1-939133-39-7. [Online]. Available: https://www.usenix.org/conference/nsdi24/presentation/hsieh

[17] W. Villegas, J. Govea, A. Navarro, and P. Palacios, "Intrusion detection in iot networks using dynamic graph modeling and graph-based neural networks," *IEEE Access*, vol. PP, pp. 1–1, Jan. 2025. DOI: 10.1109/ACCESS.2025.3559325

[18] M. Termos, Z. Ghalmane, M.-E.-A. Brahmia, A. Fadlallah, A. Jaber, and M. Zghal, "GDLC: A new graph deep learning framework based on centrality

measures for intrusion detection in iot networks," *Internet of Things*, vol. 26, p. 101 214, 2024. DOI: 10.1016/j.iot.2024.101214

[19] M. Adil, M. A. Jan, S. B. Hakim, H. H. Song, and Z. Jin, *Xids-ensembleguard: An explainable ensemble learning-based intrusion detection system*, 2025. arXiv: 2503.00615 [cs.CR]. Accessed: Jun. 3, 2026. [Online]. Available: https://arxiv.org/abs/2503.00615

[20] S. Nugroho, "A stacking ensemble learning framework for network intrusion detection using xgboost, lightgbm, catboost, and adaboost with random forest meta model," in *2026 International Conference on Communication Networks and Machine Learning (CNML)*, Jan. 2026, pp. 20–23. DOI: 10.1109/CNML68938.2026.11452388

[21] J. Zhu and X. Liu, "An integrated intrusion detection framework based on subspace clustering and ensemble learning," *Computers and Electrical Engineering*, vol. 115, p. 109 113, 2024. DOI: 10.1016/j.compeleceng.2024.109113

[22] P. R. C. Rani and K. Baalaji, "Deep learning-based ensemble stacking for enhanced intrusion detection in iot-edge platforms," *Discover Applied Sciences*, vol. 7, no. 8, p. 928, 2025. DOI: 10.1007/s42452-025-06871-z

[23] N. Moustafa and J. Slay, "Unsw-nb15: A comprehensive data set for network intrusion detection systems," *Military Communications and Information Systems Conference*, pp. 1–6, 2015. DOI: 10.1109/MilCIS.2015.7348942

[24] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy*, 2018, pp. 108–116. DOI: 10.5220/0006639801080116

[25] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-iot dataset," *Future Generation Computer Systems*, vol. 100, pp. 779–796, 2019. DOI: 10.1016/j.future.2019.05.041

[26] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, "Ton_iot telemetry dataset: A new generation dataset of iot and iiot for data-driven intrusion detection systems," *IEEE Access*, vol. 8, pp. 165 130–165 150, 2020. DOI: 10.1109/ACCESS.2020.3022862

[27] C. Yin, Y. Zhu, J. Fei, and X. He, "A deep learning approach for intrusion detection using recurrent neural networks," *IEEE Access*, vol. 5, pp. 21 954–21 961, 2017. DOI: 10.1109/ACCESS.2017.2762418

[28] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol. 7, pp. 41 525–41 550, 2019. DOI: 10.1109/ACCESS.2019.2895334

[29] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations*, 2017. [Online]. Available: https://arxiv.org/abs/1609.02907

[30] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://arxiv.org/abs/1706.02216

[31] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, "A comprehensive survey on graph neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, pp. 4–24, 2021. DOI: 10.1109/TNNLS.2020.2978386

[32] G. Ke et al., "Lightgbm: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[33] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "Catboost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[34] Jorgecardete, *Convolutional neural networks: A comprehensive guide*, Medium, The Deep Hub, Feb. 2024. Accessed: Jun. 28, 2026. [Online]. Available: https://medium.com/thedeephub/convolutional-neural-networks-a-comprehensive-guide-5cc0b5eae175

[35] J. Hu, L. Shen, S. Albanie, G. Sun, and E. Wu, *Squeeze-and-excitation networks*, 2019. arXiv: 1709.01507 [cs.CV]. Accessed: Jun. 3, 2026. [Online]. Available: https://arxiv.org/abs/1709.01507

[36] N. Klingler, *Squeeze and excitation networks: A performance upgrade*, viso.ai Blog, viso.ai, Jul. 2024. Accessed: Jun. 28, 2026. [Online]. Available: https://viso.ai/deep-learning/squeeze-and-excite-networks/

[37] Anishnama, *Understanding bidirectional lstm for sequential data processing*, Medium, Medium, May 2023. Accessed: Jun. 28, 2026. [Online]. Available: https://medium.com/@anishnama20/understanding-bidirectional-lstm-for-sequential-data-processing-b83d6283befc

[38] E. Pichka, *Graph attention networks paper explained with illustration and pytorch implementation*, Ebrahim Pichka Blog, Ebrahim Pichka Blog, Jul. 2023. Accessed: Jun. 28, 2026. [Online]. Available: https://epichka.com/blog/2023/gat-paper-explained/

[39] T. K. Rusch, M. M. Bronstein, and S. Mishra, *A survey on oversmoothing in graph neural networks*, 2023. arXiv: 2303.10993 [cs.LG]. Accessed: Jun. 3, 2026. [Online]. Available: https://arxiv.org/abs/2303.10993

[40] Z. Goldfeld and Y. Polyanskiy, *The information bottleneck problem and its applications in machine learning*, 2020. arXiv: 2004.14941 [cs.LG]. Accessed: Jun. 3, 2026. [Online]. Available: https://arxiv.org/abs/2004.14941

[41] S. Brody, U. Alon, and E. Yahav, *How attentive are graph attention networks?* 2022. arXiv: 2105.14491 [cs.LG]. Accessed: Jun. 3, 2026. [Online]. Available: https://arxiv.org/abs/2105.14491

[42] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90

[43] J. Gama, I. Zliobaite, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014. DOI: 10.1145/2523813